я хочу разработать проект DEX Агрегатор это смарт-контракт, который помогает найти лучший способ обменять одни криптовалюты на другие, как приложение для сравнения цен в разных магазинах, но для крипто и мне нужно уточнять и узнавать некоторые детали я тебе буду задавать вопросы а ты пиши теоритическое обоснование а потом как это обычно реализуется в коде на solidity 0.8.24 и open zeppelin 5.0+

Thought for 10 seconds

Пользователь хочет разработать DEX-агрегатор и будет задавать вопросы по теоретическим и практическим аспектам. Мне нужно собрать релевантную информацию о архитектуре, протоколах, алгоритмах маршрутизации и особенностях реализации на Solidity 0.8.24 с OpenZeppelin 5.0+. Я выполню несколько параллельных поисковых запросов, чтобы охватить все ключевые темы.

Found 77 web pages

После выполнения параллельных поисковых запросов, я получил несколько релевантных результатов. Теперь мне нужно проанализировать их, чтобы подготовить основу для ответов на вопросы пользователя. Я открою несколько наиболее многообещающих результатов, чтобы получить более подробную информацию.

Read 9 pages

Aggregator contracts development - Build DeFi Solutions: Smart Contracts, DEXs & Risk Ma...

DEX Smart Contract Development: Key Considerations - IdeaSoft

加密货币交易聚合器开发实战（生产级程序代码）

How DEX Aggregators Work: A Complete Guide to Getting the...

探索8大DEX聚合器：推動加密市場效率與流動性的引擎 | Gate 新聞 View All

Теперь у меня есть достаточно информации для подготовки ответа. Я буду использовать следующие источники: StudyRaid (архитектура), IdeaSoft (особенности разработки), Xingzhi (реализация на практике), Portals.fi (руководство по агрегаторам), Gate News (обзор 1inch), ChainCatcher (архитектура 1inch), Tencent Cloud (уроки по DEX), OneKey (ParaSwap), Uniswap Blog (голландские аукционы). Я подготовлю ответ, в котором опишу теоретические основы DEX-агрегатора, включая его цель, архитектуру, алгоритмы маршрутизации, механизмы защиты от MEV и атак, а затем приведу примеры кода на Solidity 0.8.24 с OpenZeppelin 5.0+.

Звучит как интересный и комплексный проект. Конечно, я с радостью помогу. В

своих ответах я буду стараться придерживаться такой структуры: сначала **теоретическое обоснование**, а затем **практическая реализация** с примерами кода на `Solidity 0.8.24` и `OpenZeppelin 5.0+`.

Для начала, давай обозначим ключевые принципы и компоненты, которые лежат в основе любого современного DEX-агрегатора. Это будет нашей отправной точкой.

## 🧩 Теоретическое обоснование: Архитектура DEX-агрегатора

Цель DEX-агрегатора — найти и выполнить лучший маршрут для обмена токенов, сканируя ликвидность на множестве децентрализованных бирж (DEX) [30]. В современной DeFi-среде ликвидность сильно фрагментирована по разным протоколам (Uniswap, Curve, Balancer и т.д.), и агрегатор решает эту проблему, автоматизируя поиск оптимальной сделки [30].

**Ключевые компоненты архитектуры:**

1. **Off-chain роутер (Routing Engine):** Это "мозг" агрегатора, который работает вне блокчейна. Он собирает данные о ликвидности из разных источников в реальном времени и с помощью сложных алгоритмов просчитывает оптимальный путь для обмена [4] [30].
2. **On-chain смарт-контракт (Router Contract):** Это "рука" агрегатора, развернутая в блокчейне. Его задача — выполнить уже готовый маршрут, переданный off-chain роутером. Он отвечает за безопасную передачу токенов и взаимодействие с пулами ликвидности [2] [4].
3. **Алгоритм поиска пути (Pathfinding Algorithm):** Это математическая логика, которая решает, какой маршрут является лучшим. Задача сводится к поиску пути в графе, где токены — это узлы, а пулы ликвидности — это ребра с определенным "весом" (цена, комиссия, глубина ликвидности) [4].
   - **Прямые маршруты:** Обмен токена A на токен B напрямую в одном пуле.
   - **Мульти-хоп маршруты (Multi-hop):** Обмен через цепочку промежуточных токенов (например, A → C → B), если это выгоднее прямого обмена [4] [30].
   - **Сплит-маршруты (Split routes):** Дробление крупной сделки на несколько частей, которые идут по разным маршрутам, чтобы минимизировать влияние

частей, которые идут по разным маршрутам, чтобы минимизировать влияние на цену (price impact)[30] [31].

## 🛡️ Проблемы безопасности: MEV и манипуляции

Разработка агрегатора невозможна без учета ключевых угроз безопасности:

1. **MEV (Maximal Extractable Value):** Боты и валидаторы могут отслеживать транзакции в мемпуле и вставлять свои сделки до или после вашей (фронтраннинг, сэндвич-атаки), чтобы получить прибыль за счет пользователя.
   - **Защита:** Использование механизмов вроде "голландских аукционов", приватного потока ордеров (private order flow) или пакетных аукционов (batch auctions)[69].
2. **Манипуляции ценами и атаки с флеш-кредитами:** Злоумышленники могут использовать моментальные займы (flash loans), чтобы временно изменить цену в пуле и провести сделку по невыгодному для пула курсу[54].
   - **Защита:** Получение ценовых данных из децентрализованных оракулов (например, Chainlink) или использование механизмов TWAP (Time-Weighted Average Price), которые усредняют цену за определенный период[54].

## 💻 Практическая реализация: Основы на Solidity

Теперь перейдем к коду. Основой нашего агрегатора будет смарт-контракт-роутер. Вот пример его базовой структуры с использованием современных практик и библиотек.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24; // Используем версию Solidity, совместимую с OpenZeppelin 5.0+

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
```

```
/**
 * @title DEXAggregator
 * @dev Базовый контракт роутера, выполняющий готовые маршруты обмена
 */
contract DEXAggregator is ReentrancyGuard, Ownable {
    using SafeERC20 for IERC20;

    // Событие, которое будет логировать каждый выполненный обмен
    event Swapped(
        address indexed user,
        address indexed tokenIn,
        address indexed tokenOut,
        uint256 amountIn,
        uint256 amountOut
    );

    // Конструктор, устанавливающий владельца контракта
    constructor() Ownable(msg.sender) {}

    /**
     * @dev Основная функция для выполнения обмена.
     * @param route Зашифрованные данные о маршруте, подготовленные off-chain.
     * @param amountIn Количество токенов, которые пользователь хочет обменять.
     * @param minAmountOut Минимальное количество токенов, которое пользователь
     *                     готов получить (защита от проскальзывания).
     */
    function swap(
        bytes calldata route,
        uint256 amountIn,
        uint256 minAmountOut
    ) external nonReentrant {
        // 1. Декодируем маршрут, полученный от off-chain сервиса
        //    (В реальном проекте здесь будет сложная логика декодирования)
        (address[] memory path, address[] memory pools) = abi.decode(route, (addre
ss[], address[]));

        // 2. Переводим токены от пользователя на контракт
        IERC20(path[0]).safeTransferFrom(msg.sender, address(this), amountIn);

        // 3. Выполняем обмен по переданному маршруту
        uint256 amountOut = _executeSwap(path, pools, amountIn);

        // 4. Проверяем, что полученное количество токенов не меньше минимально до
```

```
пустимого
        require(amountOut >= minAmountOut, "DEXAggregator: Slippage too high");

        // 5. Отправляем полученные токены пользователю
        IERC20(path[path.length - 1]).safeTransfer(msg.sender, amountOut);

        // 6. Логируем успешный обмен
        emit Swapped(msg.sender, path[0], path[path.length - 1], amountIn, amountO
ut);
    }

    /**
     * @dev Внутренняя функция, которая проходит по маршруту и вызывает пулы.
     * @param path Массив адресов токенов в пути обмена.
     * @param pools Массив адресов пулов ликвидности для каждого шага.
     * @param amount Количество токенов для первого шага.
     * @return amountOut Количество токенов, полученных в итоге.
     */
    function _executeSwap(
        address[] memory path,
        address[] memory pools,
        uint256 amount
    ) internal returns (uint256) {
        uint256 amountIn = amount;
        // Проходим по каждому шагу маршрута (например, [USDT, WETH, DAI])
        for (uint256 i = 0; i < path.length - 1; i++) {
            // Здесь вызывается функция swap на конкретном пуле ликвидности.
            // В реальности пулы разных DEX (Uniswap, Curve) имеют разные интерфей
сы,
            // поэтому потребуется более сложная логика вызова.
            amountIn = _swapOnPool(pools[i], path[i], path[i + 1], amountIn);
        }
        return amountIn;
    }

    /**
     * @dev Функция-заглушка для взаимодействия с пулом ликвидности.
     *      В реальном проекте здесь будет вызов функции swap конкретного DEX.
     */
    function _swapOnPool(
        address pool,
        address tokenIn,
        address tokenOut,
        uint256 amountIn
```

```
    ) internal returns (uint256 amountOut) {
        // 1. Даем аппрув (allowance) пулу на использование токенов tokenIn
        IERC20(tokenIn).forceApprove(pool, amountIn);

        // 2. Вызываем функцию swap у пула
        //    Пример для Uniswap V2:
        //    (uint256 reserveIn, uint256 reserveOut) = IUniswapV2Pair(pool).getRe
serves();
        //    amountOut = getAmountOut(amountIn, reserveIn, reserveOut);
        //    IUniswapV2Pair(pool).swap(0, amountOut, address(this), new bytes
(0));
        //    В данном примере оставим простой возврат для демонстрации.
        amountOut = amountIn; // ЗАГЛУШКА! Замените на реальный вызов пула!
    }

    /**
     * @dev Функция для вывода случайно отправленных на контракт токенов (только д
ля владельца).
     */
    function rescueTokens(address token) external onlyOwner {
        IERC20(token).safeTransfer(owner(), IERC20(token).balanceOf(address(thi
s)));
    }
}
```

## 📝 Ключевые моменты и лучшие практики в коде

- **Solidity 0.8.24 и OpenZeppelin 5.0+:** Используем актуальные версии. В OpenZeppelin 5.0 были удалены "хуки" (hooks) из ERC-20/721/1155 для упрощения и повышения безопасности, а также добавлены другие улучшения.
- `ReentrancyGuard`: Наследование от этого контракта OpenZeppelin защищает функцию `swap` от атак повторного входа (reentrancy attacks) с помощью модификатора `nonReentrant` [54].
- `SafeERC20`: Библиотека `SafeERC20` и её функции `safeTransferFrom`, `safeTransfer` и `forceApprove` — это критически важный инструмент. Она гарантирует, что вызовы ERC-20 токенов не приведут к неожиданным ошибкам, и обрабатывает токены, которые не возвращают `bool` при `transfer` [54].
- **Защита от проскальзывания (Slippage):** Параметр `minAmountOut` — это

обязательный элемент. Он позволяет пользователю указать минимальное приемлемое количество токенов на выходе. Если рынок изменится за время обработки транзакции и курс станет хуже, транзакция будет отменена ( `revert` ), защищая средства пользователя[54].

- **Гибкий маршрут:** Параметр `route` (тип `bytes` ) позволяет передавать в контракт произвольные маршруты, сгенерированные off-chain роутером. Это ключевой архитектурный паттерн, который отделяет сложную логику поиска от исполнения.

Это базовая структура, с которой можно начинать. Задавай любые вопросы — чем детальнее, тем лучше. Можем обсудить алгоритмы маршрутизации, интеграцию с конкретными DEX, механизмы защиты от MEV или оптимизацию газа. Жду твоих вопросов!

77 web pages

объясни формулу константного произведения (x*y=k) в Uniswap V2 и как она гарантирует ликвидность?

Thought for 5 seconds

> Мыслим по шагам. Пользователь ранее просил теоретическое обоснование и пример кода для DEX-агрегатора. Теперь задает конкретный вопрос о формуле константного произведения в Uniswap V2 и как она гарантирует ликвидность. Нужно дать понятное объяснение: формула x*y=k, как она работает при обмене, почему обеспечивает ликвидность и отсутствие исчерпания пула. Также можно упомянуть практическую реализацию расчетов в контексте Solidity и агрегатора. Ответ должен сочетать теорию с кодом (как обычно реализуется в Solidity). Пользователь ожидает объяснения именно для понимания механики AMM.

Отличный вопрос. Это сердце любого Automated Market Maker (AMM). В контексте DEX-агрегатора понимание этой механики критически важно, потому что агрегатор должен уметь вычислять точное количество токенов на выходе из пула Uniswap V2, не делая запросов к блокчейну на каждый чих.

## 🧪 Теоретическое обоснование: Магия постоянного произведения

Формула `x * y = k` — это математическая модель, которая заменяет классический стакан заявок (order book) на централизованной бирже.

1. **Определения:**
   - `x` — это резерв (количество) **Токена А** в пуле ликвидности.
   - `y` — это резерв (количество) **Токена Б** в пуле ликвидности.
   - `k` — это **Константа**. Она должна оставаться неизменной **в процессе одного обмена** (до учета комиссии).
2. **Как это работает на пальцах?**
   Представь весы, которые всегда должны находиться в равновесии. Если ты забираешь с одной чаши часть груза ( `Δy` — ты хочешь купить Токен Б), ты **обязан** положить на другую чашу некоторый груз ( `Δx` — ты платишь Токеном А), чтобы весы не накренились (чтобы `k` осталось прежним).

   На самом деле, когда ты добавляешь Токен А в пул, `x` становится больше ( `x + Δx` ). Чтобы произведение осталось равным `k` , резерв Токена Б **должен** уменьшиться до величины `y - Δy` .
3. **Почему это гарантирует ликвидность?**
   Ключевое свойство этой кривой (гиперболы) в том, что у нее **нет экстремумов**.
   - **Защита от полного опустошения:** Чем больше Токена Б ты пытаешься вытащить (т.е. чем ближе `y` к нулю), тем экспоненциально больше Токена А тебе придется заплатить. Цена уходит в бесконечность.
   - **Гарантия исполнения:** В отличие от ордербука, где может просто не быть встречной заявки, здесь **всегда** есть цена, по которой можно обменять хоть микроскопическую сумму, хоть половину пула. Сделка состоится всегда, вопрос лишь в цене.
4. **Роль комиссии и изменение `k` .**
   В реальности `k` не константа навсегда. Протокол берет комиссию (обычно 0.3%).
   - **Без комиссии:** `(x + Δx) * (y - Δy) = x * y` .
   - **С комиссией:** 0.3% от `Δx` **остается** в пуле. То есть пул получает `Δx` , а в расчет берется `0.997 * Δx` .
   - Формула с комиссией: `(x + 0.997 * Δx) * (y - Δy) = x * y`

- Формула с комиссией: `(x + 0.997 * dx) * (y - dy) = x * y`.
- В результате `x_new * y_new > x * y`. `k` **растет**, увеличивая богатство поставщиков ликвидности.

## 💻 Практическая реализация: Функции getAmountOut в коде агрегатора

Когда мы пишем DEX-агрегатор, мы не можем для каждого маршрута вызывать контракт пула, чтобы спросить цену (это потратит весь газ). Мы должны просчитать результат **локально** в нашем смарт-контракте. Для этого нам нужны точные формулы Uniswap V2.

Вот как это реализуется в **Solidity 0.8.24** внутри библиотеки или контракта-агрегатора:

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

library UniswapV2Library {
    /**
     * @dev Расчет количества Токена Б, которое мы получим, отдав amountIn Токена А.
     * Использует формулу Uniswap V2 с комиссией 0.3%.
     *
     * @param amountIn Количество токенов, которое мы отдаем.
     * @param reserveIn Резерв входящего токена в пуле (x).
     * @param reserveOut Резерв исходящего токена в пуле (y).
     * @return amountOut Количество токенов на выходе.
     */
    function getAmountOut(
        uint256 amountIn,
        uint256 reserveIn,
        uint256 reserveOut
    ) internal pure returns (uint256 amountOut) {
        // 1. Валидация: резервы должны быть > 0, иначе деление на ноль
        require(amountIn > 0, "UniswapV2Library: INSUFFICIENT_INPUT_AMOUNT");
        require(reserveIn > 0 && reserveOut > 0, "UniswapV2Library: INSUFFICIENT_LIQUIDITY");
```

```
        // 2. Учитываем комиссию протокола (0.3%).
        //    В блокчейне нет плавающей точки, поэтому работаем с целыми числами.
        //    amountInWithFee = amountIn * 997
        uint256 amountInWithFee = amountIn * 997;

        // 3. Считаем числитель (верхняя часть дроби)
        //    numerator = amountInWithFee * reserveOut
        uint256 numerator = amountInWithFee * reserveOut;

        // 4. Считаем знаменатель (нижняя часть дроби)
        //    denominator = (reserveIn * 1000) + amountInWithFee
        //    Это вытекает из формулы: (x + 0.997*dx) * (y - dy) = x*y
        //    => dy = (0.997*dx * y) / (x + 0.997*dx)
        uint256 denominator = (reserveIn * 1000) + amountInWithFee;

        // 5. Выполняем целочисленное деление (округление вниз в пользу пула)
        amountOut = numerator / denominator;
    }

    /**
     * @dev Обратная задача: сколько Токена А нужно отдать, чтобы получить amountO
ut Токена Б.
     * Часто используется для расчета "minAmountOut" в агрегаторах.
     */
    function getAmountIn(
        uint256 amountOut,
        uint256 reserveIn,
        uint256 reserveOut
    ) internal pure returns (uint256 amountIn) {
        require(amountOut > 0, "UniswapV2Library: INSUFFICIENT_OUTPUT_AMOUNT");
        require(reserveIn > 0 && reserveOut > 0, "UniswapV2Library: INSUFFICIENT_L
IQUIDITY");

        // Формула: dx = (x * dy) / ( (y - dy) * 0.997 )
        uint256 numerator = reserveIn * amountOut * 1000;
        uint256 denominator = (reserveOut - amountOut) * 997;

        // Добавляем 1 для корректного округления вверх (чтобы точно хватило на am
ountOut)
        amountIn = (numerator / denominator) + 1;
    }
}
```

## 🔗 Как это встраивается в логику агрегатора?

В функции `_swapOnPool` из моего предыдущего ответа мы бы использовали эту библиотеку так:

```solidity
function _swapOnPool(
    address pool,
    address tokenIn,
    address tokenOut,
    uint256 amountIn
) internal returns (uint256 amountOut) {
    // 1. Получаем текущие резервы из пула (это можно сделать за 1 вызов)
    (uint256 reserve0, uint256 reserve1, ) = IUniswapV2Pair(pool).getReserves();

    // 2. Определяем, какой резерв соответствует какому токену
    address token0 = IUniswapV2Pair(pool).token0();
    (uint256 reserveIn, uint256 reserveOut) = tokenIn == token0
        ? (reserve0, reserve1)
        : (reserve1, reserve0);

    // 3. ВЫЧИСЛЯЕМ amountOut ЛОКАЛЬНО, используя нашу библиотеку
    amountOut = UniswapV2Library.getAmountOut(amountIn, reserveIn, reserveOut);

    // 4. Выполняем своп. Обрати внимание: мы не передаем amountOut в пул,
    //    мы говорим пулу: "Я даю тебе amountIn, верни мне минимум amountOut".
    //    Пул САМ внутри себя посчитает amountOut по той же формуле, что и мы.
    IUniswapV2Pair(pool).swap(
        tokenIn == token0 ? 0 : amountOut, // amount0Out
        tokenIn == token0 ? amountOut : 0, // amount1Out
        address(this),                     // recipient
        new bytes(0)                       // data
    );
}
```

## 💡 Важный нюанс: Атаки на основе резервов

Именно эта формула является причиной, по которой DEX-агрегатор **не должен** полагаться только на локальный расчет `getAmountOut` с "грязными" резервами. Если злоумышленник видит, что агрегатор рассчитывает сделку на основе

резервов блока `N`, а исполнение идет в блоке `N+1`, он может изменить эти резервы в своем блоке и подставить агрегатор (sandwich attack). Поэтому в реальных агрегаторах (как 1inch или Matcha) используют **оффчейн-симуляции** на основе последнего состояния мемпула и **защиту от проскальзывания** через `minAmountOut`.

чем отличается Uniswap V2 от Uniswap V3 картко и что такое concentrated liquidity

Thought for 5 seconds

> Пользователь спрашивает о различиях между Uniswap V2 и Uniswap V3, а также о концепции концентрированной ликвидности. Нужно дать краткое объяснение. Возможно, стоит поискать информацию.

Found 16 web pages

> Чтобы получить более полную информацию, я открою несколько наиболее релевантных результатов.

Read 4 pages Uniswap v2 vs. v3: A 2025 Deep Dive ↗
Chapter 5 - How does Concentrated Liquidity (CL) work | GammaSwap Academy ↗
What is Uniswap V3? Concentrated Liquidity & Fee Tiers | Nansen ↗
Liquidity Positions | Uniswap ↗

> Теперь у меня есть достаточно информации, чтобы ответить пользователю. Я объясню различия между Uniswap V2 и V3, а также концепцию концентрированной ликвидности. Для наглядности приведу таблицу и пример с кодом.

**Uniswap V3** — это не просто апдейт V2, а глубокая переработка модели работы с ликвидностью. Её главная задача — решить проблему **неэффективности капитала**. В то время как в V2 ликвидность была размазана по всему бесконечному ценовому диапазону и по большей части простаивала, V3 позволяет точечно размещать капитал там, где он реально нужен.

Ниже — сжатое сравнение и подробный разбор ключевого нововведения.

## 📊 Uniswap V2 vs. Uniswap V3: Ключевые отличия

| Характеристика | Uniswap V2 | Uniswap V3 |
|---|---|---|

| Характеристика | Uniswap V2 | Uniswap V3 |
|---|---|---|
| **Модель ликвидности** | Равномерное распределение по всей кривой от 0 до бесконечности [6] [1]. | **Концентрированная**: LPs сами выбирают ценовой диапазон для своего капитала [11]. |
| **Капиталоэффективность** | **Низкая**. Большая часть ликвидности в пуле никогда не используется для сделок [1]. | **Высокая**. Прирост эффективности может достигать **4000 раз** по сравнению с V2 [11]. |
| **Представление позиции LP** | **Взаимозаменяемые** токены (ERC-20) [6]. | **Невзаимозаменяемые** токены (ERC-721 NFT), уникальные для каждого диапазона и стратегии [11]. |
| **Комиссии** | Фиксированная ставка **0.30%** для всех пулов [6]. | Гибкие уровни: **0.05%**, **0.30%**, **1%** (и другие), в зависимости от волатильности пары [6]. |
| **Сложность для LP** | **Низкая**. Стратегия "поставил и забыл" (set-and-forget) [6]. | **Высокая**. Требует активного управления диапазоном и рисками [6]. |
| **Сложность для агрегатора** | **Низкая**. Все пулы используют единую и простую формулу `x*y=k`. | **Высокая**. Нужно учитывать множество пулов с разными комиссиями и, что критично, динамически рассчитывать ликвидность в каждом тике. |

## 🎯 Что такое концентрированная ликвидность (Concentrated Liquidity)?

Это фундаментальная инновация Uniswap V3, которая позволяет поставщикам ликвидности (LPs) не распылять свой капитал, а **сконцентрировать его в узком**

**ценовом коридоре**, где, по их мнению, будет происходить наибольшая торговая активность.

**Как это выглядит на практике?**

Представьте, что вы предоставляете ликвидность в паре ETH/USDC и считаете, что цена ETH в ближайшее время будет колебаться в диапазоне $1500-$2500.

- **В Uniswap V2**: Ваши ETH и USDC будут "размазаны" по всей бесконечной шкале цен. Это как расставить товар по всему футбольному полю: 99% его никто не увидит и не купит [11].
- **В Uniswap V3**: Вы можете "разместить" свой капитал только на участке от $1500 до $2500. Внутри этого диапазона ваша ликвидность становится **суперконцентрированной**, как если бы вы выставили весь товар в самой проходимой точке стадиона. Как только цена выходит за эти пределы, ваша ликвидность "выключается" и перестаёт приносить комиссии до тех пор, пока цена не вернётся [1].

> **Математическая аналогия:** Uniswap V3 работает как бесконечное множество Uniswap V2 пулов, где каждый из них "живёт" только в своём узком ценовом диапазоне (тике) [1].

**Плюсы для LPs и трейдеров**

- **Для LPs:** Возможность получать значительно больше комиссий с меньшим капиталом за счёт фокусировки на волатильности.
- **Для трейдеров: Гораздо лучшие цены и меньшее проскальзывание** (slippage), особенно для крупных сделок. Поскольку ликвидность сконцентрирована вокруг текущей цены, глубина рынка становится намного больше [12].

## 💻 Как это меняет код DEX-агрегатора

Когда вы разрабатываете агрегатор, который должен работать с Uniswap V3, логика кардинально усложняется. Вместо того чтобы просто запросить резервы пула и подставить их в формулу `getAmountOut` (как мы делали в прошлом ответе для V2), вам нужно делать следующее:

```solidity
// Упрощённый пример для демонстрации концепции

// Вместо простого вызова функции getAmountOut для всего пула...
// function _swapOnUniswapV2(address pool, uint256 amountIn) internal returns (uin
t256) {
//    (uint112 reserve0, uint112 reserve1, ) = IUniswapV2Pair(pool).getReserves();
//    return UniswapV2Library.getAmountOut(amountIn, reserve0, reserve1);
// }

// ...вам нужно итерироваться по "тикам", чтобы выяснить,
// сколько ликвидности доступно по текущей и последующим ценам.
function _swapOnUniswapV3(address pool, bool zeroForOne, uint256 amountIn) interna
l returns (uint256) {
    // 1. Получаем текущее состояние пула: цену (sqrtPriceX96), индекс тика (tick)
и ликвидность (liquidity).
    (uint160 sqrtPriceX96, int24 currentTick, , , , , ) = IUniswapV3Pool(pool).slo
t0();

    // 2. Основной цикл свопа: проходим по тикам, пока не исчерпаем amountIn.
    while (amountIn > 0) {
        // Определяем следующий инициализированный тик (границу ликвидности) в нап
равлении сделки.
        // Для этого нужно изучить тик-битмап (tickBitmap) пула.
        (int24 nextTick, bool initialized) = _getNextTick(pool, currentTick, zeroF
orOne);

        // Рассчитываем, какое количество входящего токена потребуется, чтобы цена
дошла до nextTick.
        uint256 amountInToNextTick = _calculateAmountInToNextTick(amountIn, sqrtPr
iceX96, liquidity, nextTick);

        // Выполняем обмен до следующего тика...
        // ...и обновляем amountIn, sqrtPriceX96, currentTick и liquidity.
    }
}
```

### Почему это важно?

Агрегатор больше не может полагаться на простую формулу для одного пула. Чтобы найти лучший маршрут, он должен:

1. **Собрать данные** обо всех активных тиках и распределении ликвидности в них

для каждого V3 пула.

2. **Смоделировать** прохождение сделки через эту "дискретную" ликвидность.
3. **Сравнить** результаты с другими V3 пулами, V2 пулами и другими источниками ликвидности.

Это делает вычисления на порядок сложнее, поэтому зачастую они выполняются вне блокчейна (off-chain), а в контракт агрегатора передаётся уже готовый, просчитанный маршрут.

Надеюсь, это сравнение и разбор концентрированной ликвидности помогли прояснить картину. Это ключевое различие, которое напрямую влияет на архитектуру любого современного DEX-агрегатора.

Есть ли какие-то конкретные аспекты, которые стоит разобрать более детально?

16 web pages

как рассчитывается price impact при своп-операции

Thought for 2 seconds

> Мы должны объяснить, что такое price impact (ценовое воздействие), как оно возникает в AMM, как рассчитывается для разных версий Uniswap и почему это важно для агрегатора. Нужно дать теоретическое обоснование и практические формулы с примерами кода на Solidity 0.8.24. Также упомянуть разницу между slippage и price impact.

Отличный вопрос. Понимание **Price Impact** критически важно для разработки DEX-агрегатора, потому что именно этот параметр отличает «хороший» маршрут от «убыточного». Многие новички путают *Price Impact* и *Slippage*, поэтому начнем с четкого разделения понятий.

## 🎯 Теоретическое обоснование: Price Impact vs. Slippage

| Параметр | Определение | Кто виноват? |
|---|---|---|
| **Price Impact** | Это **ожидаемое** изменение цены актива в результате **вашей конкретной сделки**. Это | **Вы сами** (размером вашей сделки относительно |

| | | |
|---|---|---|
| **Price Impact** | **конкретной сделки**. Это математическое следствие кривой AMM. | сделки относительно ликвидности пула). |
| **Slippage** | Это **разница** между ценой, которую вы ожидали увидеть в момент отправки транзакции, и ценой, по которой сделка **фактически** исполнилась в блокчейне. | **Другие трейдеры, боты, время** (изменение состояния пула между подписанием и майнингом). |

**Простая аналогия:**
Вы хотите купить 1000 арбузов на рынке.

- **Price Impact:** Первый арбуз стоит 1$, но к тому моменту, как вы скупите 1000 штук, продавец поймет ажиотаж и поднимет цену последнего арбуза до 2$. Это неизбежное следствие вашего аппетита.
- **Slippage:** Вы запланировали купить 1000 арбузов по средней цене 1.5$, но пока вы шли к прилавку, прибежал другой покупатель и скупил 500 штук до вас. Теперь средняя цена для вас стала 1.8$. Вот это slippage.

## 🧮 Как рассчитать Price Impact в Uniswap V2 (и почему он всегда есть)

В Uniswap V2 используется формула `x * y = k`. Давайте выведем формулу Price Impact математически.

**Исходные данные:**

- Резерв Токена А (например, USDC): `X`
- Резерв Токена Б (например, ETH): `Y`
- Начальная цена (спот): `P0 = Y / X` (Сколько ETH дают за 1 USDC).

**Совершаем сделку:**
Мы продаем `dx` количество USDC и получаем `dy` количество ETH.
Согласно формуле `(X + dx) * (Y - dy) = X * Y`.
Отсюда количество полученного ETH:

```
dy = (Y * dx) / (X + dx)
```

**Эффективная цена исполнения (Execution Price):**

Это средняя цена, по которой прошла вся сделка.

```
P_exec = dy / dx = Y / (X + dx)
```

**Price Impact (в процентах):**

Это разница между начальной ценой ( `P0` ) и ценой исполнения ( `P_exec` ), деленная на начальную цену.

```
Price Impact (%) = ( (P0 - P_exec) / P0 ) * 100%
```

Подставляем значения:

```
Price Impact (%) = ( (Y/X) - (Y/(X+dx)) ) / (Y/X) = 1 - (X / (X+dx)) = dx / (X
+ dx)
```

> **Ключевой вывод для агрегатора:**
> Price Impact зависит **только** от соотношения `dx / X` (размер вашей сделки к размеру резерва пула).
>
> - Если сделка составляет 1% от пула ( `dx = 0.01X` ), Impact ≈ 0.99%.
> - Если сделка составляет 10% от пула ( `dx = 0.1X` ), Impact ≈ 9.09%.
> - Если сделка составляет 50% от пула, Impact = 33%.

## 💻 Практическая реализация расчета Price Impact в Solidity

В смарт-контракте агрегатора нам нужно уметь быстро оценить ценовое воздействие маршрута, чтобы сравнить разные варианты. Так как в Solidity нет плавающей точки, мы работаем с базисными пунктами (bps) или множителями `1e18` .

Вот пример библиотеки для расчета Price Impact на Uniswap V2 с использованием **Solidity 0.8.24**.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;
```

```
library PriceImpactCalculator {
    // Точность вычислений (18 знаков после запятой в мире целых чисел)
    uint256 constant PRECISION = 1e18;

    /**
     * @dev Рассчитывает Price Impact для свопа в Uniswap V2.
     * @param amountIn Количество входящего токена (dx).
     * @param reserveIn Резерв входящего токена в пуле (X).
     * @return impactPercentage Ценовое воздействие в процентах с точностью 1e18.
     *         (Например, 0.05e18 = 5%).
     */
    function calculatePriceImpactV2(
        uint256 amountIn,
        uint256 reserveIn
    ) internal pure returns (uint256 impactPercentage) {
        require(reserveIn > 0, "Zero reserve");

        // Формула: impact = dx / (X + dx)
        // В целых числах: impact = (amountIn * PRECISION) / (reserveIn + amountI
n)
        //
        // Пример: amountIn = 100, reserveIn = 1000
        // impact = 100 / 1100 = 0.0909 (9.09%)
        impactPercentage = (amountIn * PRECISION) / (reserveIn + amountIn);
    }

    /**
     * @dev Рассчитывает эффективную цену исполнения с учетом Price Impact.
     * @param amountIn Количество входящего токена.
     * @param reserveIn Резерв входящего токена.
     * @param reserveOut Резерв исходящего токена.
     * @return executionPrice Цена исполнения (amountOut / amountIn) * PRECISION.
     */
    function calculateExecutionPriceV2(
        uint256 amountIn,
        uint256 reserveIn,
        uint256 reserveOut
    ) internal pure returns (uint256 executionPrice) {
        // amountOut = (amountIn * 997 * reserveOut) / (reserveIn * 1000 + amountI
n * 997)
        uint256 amountInWithFee = amountIn * 997;
        uint256 numerator = amountInWithFee * reserveOut;
        uint256 denominator = (reserveIn * 1000) + amountInWithFee;
        uint256 amountOut = numerator / denominator;
```

```
        // Цена = amountOut / amountIn
        executionPrice = (amountOut * PRECISION) / amountIn;
    }

    /**
     * @dev Альтернативный расчет Impact через сравнение спотовой цены и цены испо
лнения.
     * Полезен для сложных маршрутов (мульти-хоп).
     */
    function calculatePriceImpactFromPrices(
        uint256 spotPrice,      // Текущая цена спот (Y/X) * PRECISION
        uint256 executionPrice  // Цена сделки * PRECISION
    ) internal pure returns (uint256 impact) {
        // impact = (spotPrice - executionPrice) / spotPrice
        // Если executionPrice ниже (мы купили дешевле, чем в реальности, такого н
е бывает в AMM),
        // но для продажи цена будет ниже.
        // Для покупки токена (мы тратим X, получаем Y): executionPrice всегда < s
potPrice
        // impact = 1 - (executionPrice / spotPrice)
        impact = PRECISION - ((executionPrice * PRECISION) / spotPrice);
    }
}
```

## 🧩 Как это используется в логике DEX-агрегатора?

В контракте агрегатора вы можете использовать этот расчет в модификаторе или во view-функции для валидации маршрута перед отправкой транзакции пользователем (например, во фронтенде или в симуляции).

```solidity
// Пример функции симуляции свопа (view)
function simulateSwap(
    bytes calldata route,
    uint256 amountIn
) external view returns (uint256 amountOut, uint256 priceImpact) {
    // ... декодируем маршрут, получаем массив резервов пулов ...

    // Рассчитываем ожидаемый amountOut (используя UniswapV2Library)
    //
```

```
    // ...

    // Считаем общий Price Impact по всему маршруту
    // Если маршрут сложный, impact считается итеративно или через отношение
    // начальной спотовой цены к конечной цене исполнения.

    uint256 spotPrice = getSpotPrice(path[0], path[1]); // Цена без учета сделки
    uint256 executionPrice = (amountOut * 1e18) / amountIn;

    if (executionPrice < spotPrice) {
        priceImpact = PriceImpactCalculator.calculatePriceImpactFromPrices(spotPri
ce, executionPrice);
    }

    return (amountOut, priceImpact);
}
```

## 🔮 Особенности расчета Price Impact в Uniswap V3

В Uniswap V3 с концентрированной ликвидностью расчет Price Impact становится **итеративным** и зависит от того, как сделка «проходит» через различные ценовые тики.

**Упрощенный алгоритм для оффчейн-расчетов:**

1. Берем текущий тик ( `tick` ) и ликвидность ( `L` ) внутри него.
2. Считаем, какое количество `dx` нужно, чтобы цена сдвинулась до следующего инициализированного тика.
3. Если `dx` нашей сделки больше, чем нужно для перехода в следующий тик, мы:
   - Расходуем эту часть `dx` .
   - Считаем полученное `dy` .
   - Переходим на следующий тик, обновляем `L` (добавляем/убираем ликвидность, которая «включилась/выключилась» в этом тике).
   - Повторяем.
4. Когда `dx` закончился, смотрим итоговую цену исполнения и сравниваем с начальной спотовой ценой.

внутри смарт-контракта такой цикл выполнять **слишком дорого по газу**. Поэтому агрегаторы (1inch, Matcha) делают эти вычисления **вне блокчейна** (на сервере или в браузере пользователя), а в контракт передают уже готовый массив действий для исполнения.

## ⚠️ Важное замечание про безопасность в контракте

Даже если агрегатор рассчитал `minAmountOut` с учетом допустимого `slippage`, высокий `Price Impact` — это **прямой убыток для пользователя**. В хорошем агрегаторе стоит добавить проверку:

```solidity
error PriceImpactTooHigh(uint256 impact, uint256 maxAllowed);

modifier limitPriceImpact(uint256 impact, uint256 maxImpactBps) {
    // maxImpactBps в базисных пунктах (1% = 100)
    if (impact > (maxImpactBps * 1e16)) { // 1e16 = 1% от 1e18
        revert PriceImpactTooHigh(impact, maxImpactBps);
    }
    _;
}
```

Это защитит пользователя от случайного или злонамеренного проведения сделки через "пустынный" пул, где его сделка сдвинет рынок на 20% в невыгодную сторону.

Задавай следующие вопросы — можем детальнее разобрать интеграцию конкретных протоколов или алгоритмы сплит-маршрутизации.

объясни концепцию slippage

Thought for 5 seconds

> Мы уже подробно обсуждали price impact, и в том ответе я упомянул разницу между price impact и slippage. Теперь пользователь просит объяснить концепцию slippage отдельно. Нужно дать четкое теоретическое объяснение, затем практическую реализацию в контракте агрегатора с использованием minAmountOut и, возможно, deadline. Также стоит упомянуть, как slippage влияет на пользовательский опыт и безопасность. Ответ должен

> быть на русском, структурирован: теория, потом код на Solidity 0.8.24 с OpenZeppelin.

## 🎯 Теоретическое обоснование: Slippage (Проскальзывание)

**Slippage** — это разница между **ожидаемой ценой** сделки (которую вы видели в интерфейсе кошелька до подписания транзакции) и **фактической ценой**, по которой сделка исполнилась в блокчейне.

**Ключевой нюанс:** Slippage **не связан** напрямую с кривой AMM и вашим размером сделки (это Price Impact). Slippage возникает из-за того, что **блокчейн — это асинхронная, конкурентная среда**.

Представьте: вы нашли товар в интернет-магазине по цене 1000 руб. Пока вы вводили данные карты, кто-то другой купил последний экземпляр, и магазин обновил цену до 1100 руб. Ваш заказ все равно прошел, но по новой цене. Вот это и есть slippage в мире криптовалют.

**Кто и как вызывает Slippage:**

1. **Фронтраннинг-боты (MEV):** Бот видит вашу крупную сделку в мемпуле (очереди ожидающих транзакций), моментально отправляет свою транзакцию с более высокой комиссией, сдвигает цену в пуле, а ваша сделка исполняется уже по худшей цене.
2. **Сэндвич-атаки:** Комбинация фронтраннинга и бэкраннинга. Бот покупает перед вами, вы покупаете по завышенной цене, бот продает после вас с прибылью.
3. **Обычные пользователи:** Просто высокая активность в сети. В мемпуле скопились десятки сделок в том же пуле, и ваша оказалась не первой.

**Математическое выражение Slippage:**
$Slippage = \frac{P_{expected} - P_{actual}}{P_{expected}}$
Где:

- $P_{expected}$ — цена, которую показал агрегатор в момент формирования транзакции.
- $P_{actual}$ — цена, зафиксированная в событии `Swap` после майнинга блока.

## 🛡️ Защита от Slippage в смарт-контрактах: `minAmountOut`

Единственный надежный способ защиты пользователя от негативного slippage — это параметр `minAmountOut` (или `amountOutMin`). Пользователь говорит контракту: *«Я согласен получить любое количество токенов, но **не меньше** вот этого значения. Если из-за изменений на рынке я получу меньше — отмени сделку»*.

Без этого параметра агрегатор становится "дырявым": злоумышленник может провести транзакцию пользователя через пул, где курс уже изменился на 99% в худшую сторону, и контракт этого даже не заметит.

## 💻 Практическая реализация в DEX-агрегаторе (Solidity 0.8.24)

В контракте-агрегаторе параметр `minAmountOut` проверяется в самом конце цепочки обменов. Также часто добавляют параметр `deadline` (крайний срок), чтобы старая транзакция не "зависла" в мемпуле на часы и не исполнилась по арбитражной цене.

Вот расширенная версия функции `swap` из предыдущих ответов с надежной защитой от slippage.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

contract DEXAggregatorSlippageProtection is ReentrancyGuard {
    using SafeERC20 for IERC20;

    // Кастомные ошибки (экономия газа по сравнению с require)
    error SlippageExceeded(uint256 amountOut, uint256 minAmountOut);
    error TransactionExpired(uint256 deadline, uint256 currentTimestamp);

    event Swapped(
        address indexed user,
        address indexed tokenIn,
        address indexed tokenOut
```

```
        address indexed tokenOut,
        uint256 amountIn,
        uint256 amountOut
    );

    /**
     * @dev Основная точка входа для свопа с защитой от проскальзывания и дедлайно
м.
     * @param route Закодированный маршрут.
     * @param amountIn Количество отдаваемых токенов.
     * @param minAmountOut МИНИМАЛЬНО допустимое количество получаемых токенов (за
щита от slippage).
     * @param deadline Unix timestamp, после которого транзакция считается просроч
енной.
     */
    function swap(
        bytes calldata route,
        uint256 amountIn,
        uint256 minAmountOut,
        uint256 deadline
    ) external nonReentrant {
        // 1. Проверка срока действия транзакции
        //    Защищает от ситуаций, когда транзакция застряла в мемпуле из-за низк
ой цены газа,
        //    а потом резко исполнилась, когда рынок уже ушел в другую сторону.
        if (block.timestamp > deadline) {
            revert TransactionExpired(deadline, block.timestamp);
        }

        // 2. Декодирование маршрута и перевод средств от пользователя
        (address[] memory path, address[] memory pools) = abi.decode(route, (addre
ss[], address[]));
        IERC20(path[0]).safeTransferFrom(msg.sender, address(this), amountIn);

        // 3. Выполнение мульти-хоп свопа
        uint256 amountOut = _executeMultiHopSwap(path, pools, amountIn);

        // 4. КЛЮЧЕВАЯ ПРОВЕРКА SLIPPAGE
        //    Если рынок изменился не в нашу пользу и amountOut оказался меньше ож
идаемого,
        //    транзакция откатывается (revert), и пользователь не теряет деньги.
        if (amountOut < minAmountOut) {
            revert SlippageExceeded(amountOut, minAmountOut);
        }
```

```
        // 5. Отправка результата пользователю
        IERC20(path[path.length - 1]).safeTransfer(msg.sender, amountOut);

        emit Swapped(msg.sender, path[0], path[path.length - 1], amountIn, amountO
ut);
    }

    // Вспомогательная функция выполнения мульти-хопа (заглушка)
    function _executeMultiHopSwap(
        address[] memory path,
        address[] memory pools,
        uint256 amount
    ) internal returns (uint256) {
        uint256 currentAmount = amount;
        for (uint256 i = 0; i < path.length - 1; i++) {
            // Здесь вызов конкретного пула (Uniswap V2/V3, Curve и т.д.)
            currentAmount = _swapSinglePool(pools[i], path[i], path[i+1], currentA
mount);
        }
        return currentAmount;
    }

    function _swapSinglePool(address, address, address, uint256 amountIn) internal
pure returns (uint256) {
        // Реальная логика взаимодействия с пулом
        return amountIn; // Заглушка
    }
}
```

## 📈 Как выбирать значение `minAmountOut` на фронтенде?

Это ответственность off-chain части агрегатора. Алгоритм расчета `minAmountOut` для пользователя выглядит так:

1. **Расчет идеального** `amountOut` : Просчитываем маршрут, получаем ожидаемое количество токенов на выходе (например, `1000 USDC` ).
2. **Применение допуска Slippage**: Пользователь в интерфейсе выбирает допустимый slippage (обычно 0.1%, 0.5%, 1% или "Авто").
3. **Вычисление порога**:

```javascript
// Пример расчета на JS фронтенде
const expectedAmountOut = 1000.0; // USDC
const slippagePercent = 0.5; // 0.5%
const slippageFactor = 1 - (slippagePercent / 100);
const minAmountOut = expectedAmountOut * slippageFactor; // 995.0 USDC
```

## ⚖️ Slippage vs. Price Impact: Краткий чек-лист для разработчика

| Характеристика | Price Impact | Slippage (Проскальзывание) |
|---|---|---|
| **Причина** | Размер сделки относительно ликвидности. | Временной лаг и MEV-активность. |
| **Где рассчитать?** | На основе **текущих** резервов пула. | Предсказать невозможно, только ограничить. |
| **Как защитить?** | **Сплит-маршруты**, поиск глубокой ликвидности. | `minAmountOut` и `deadline`. |
| **Валидация в контракте** | Желательна (чтобы не разорять юзера на пустом пуле). | **Обязательна** (каноническое требование безопасности). |

Если у тебя будут вопросы по конкретным сценариям (например, как правильно устанавливать `deadline` для L2 сетей или как обрабатывать реверт `SlippageExceeded` в интерфейсе кошелька), смело задавай.

> какие компоненты минимально нужны для функционирования DEX агрегатора

Thought for 11 seconds

> Мы обсуждаем DEX-агрегатор на смарт-контрактах. Вопрос: какие минимальные компоненты нужны для его функционирования. Нужно дать четкий ответ: разделить на off-

chain и on-chain части, описать ключевые модули и их назначение. Учитывая предыдущий контекст, нужно теоретическое обоснование и примеры кода на Solidity 0.8.24 с OpenZeppelin 5.0+. Пользователь хочет понять архитектурный минимум.

План ответа:

1. Теоретическое обоснование: почему минимальная архитектура именно такая (разделение на off-chain поиск маршрута и on-chain исполнение).
2. Минимально необходимые компоненты:
   - Off-chain: Aggregator Engine (сбор ликвидности, поиск пути), API для фронтенда.
   - On-chain: Router Contract (исполнение свопов), возможно вспомогательные библиотеки для расчетов (getAmountOut).
3. Пример минимального контракта-роутера с поддержкой Uniswap V2 (для простоты) с защитами.
4. Дополнительно: минимальный фронтенд компонент (скрипт расчета маршрута и отправки транзакции).

Ответ должен быть практичным и демонстрировать понимание минимального жизнеспособного продукта (MVP).

## 🧩 Теоретическое обоснование: Минимальная архитектура DEX-агрегатора

Любой DEX-агрегатор решает две принципиально разные задачи:

1. **Поиск лучшего маршрута** — вычислительно сложная задача, требующая доступа к актуальным данным о состоянии сотен пулов. Выполнять её в блокчейне **невозможно** из-за ограничений по газу и отсутствия возможности делать циклы по массивам переменной длины.
2. **Атомарное исполнение сделки** — задача, которая **должна** выполняться в блокчейне, чтобы гарантировать, что все промежуточные обмены либо произойдут вместе, либо транзакция полностью откатится.

Из этого разделения следует, что **минимально жизнеспособный DEX-агрегатор состоит из двух ключевых частей**:

- **Off-chain Routing Engine** (движок маршрутизации вне блокчейна).
- **On-chain Router Contract** (смарт-контракт-исполнитель).

Без любого из этих компонентов система неработоспособна: один контракт без

движка — это просто обычный роутер (как у Uniswap), а один движок без контракта не может выполнить сделку атомарно.

## 📦 Минимально необходимые компоненты (MVP)

| Компонент | Где находится | Задача |
|---|---|---|
| **1. Routing Engine** | Off-chain (сервер / браузер) | Собрать резервы пулов → Найт маршрут(ы) → Подготовить `bytes calldata route` |
| **2. Router Contract** | On-chain (EVM) | Принять `route` и `amountIn` → Перевести средства → Вызвать нужные пулы → Проверить `minAmountOut` → Вернуть токен |
| **3. Фронтенд** | Браузер пользователя | Запросить у движка маршрут → Сформировать транзакцию для контракта → Отправить через кошелек |
| **4. Интеграция с RPC** | Off-chain | Получение актуальных данных состоянии пулов |

## 💻 Минимальный On-chain Router Contract (Solidity 0.8.24 + OZ 5.0+)

Этот контракт умеет выполнять **только** Uniswap V2 свопы (для простоты MVP). Он принимает закодированный маршрут в виде массива адресов пулов и массива токенов, выполняет их последовательно и проверяет защиту от проскальзывания.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;
```

```
pragma solidity 0.8.24;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";

interface IUniswapV2Pair {
    function swap(uint256 amount0Out, uint256 amount1Out, address to, bytes callda
ta data) external;
    function token0() external view returns (address);
    function token1() external view returns (address);
    function getReserves() external view returns (uint112 reserve0, uint112 reserv
e1, uint32 blockTimestampLast);
}

/**
 * @title MinimalDEXAggregator
 * @dev Абсолютный минимум для агрегации Uniswap V2 пулов.
 */
contract MinimalDEXAggregator is ReentrancyGuard {
    using SafeERC20 for IERC20;

    error SlippageExceeded(uint256 received, uint256 minimum);
    error DeadlineExpired();

    event SwapExecuted(address indexed user, address tokenIn, address tokenOut, ui
nt256 amountIn, uint256 amountOut);

    /**
     * @dev Единственная точка входа для пользователя.
     * @param path Маршрут токенов: [tokenIn, intermediate1, ..., tokenOut]
     * @param pools Адреса пулов Uniswap V2 для каждого шага (длина = path.length
- 1)
     * @param amountIn Количество tokenIn, которое забирается у пользователя
     * @param minAmountOut Минимальное приемлемое количество tokenOut
     * @param deadline Таймстемп, после которого транзакция недействительна
     */
    function swap(
        address[] calldata path,
        address[] calldata pools,
        uint256 amountIn,
        uint256 minAmountOut,
        uint256 deadline
    ) external nonReentrant {
```

```
        if (block.timestamp > deadline) revert DeadlineExpired();
        require(path.length >= 2, "Invalid path");
        require(pools.length == path.length - 1, "Pools length mismatch");

        // 1. Забираем входящий токен у пользователя
        IERC20(path[0]).safeTransferFrom(msg.sender, address(this), amountIn);

        // 2. Проходим по всем шагам маршрута
        uint256 currentAmount = amountIn;
        for (uint256 i = 0; i < pools.length; i++) {
            currentAmount = _swapUniswapV2(
                pools[i],
                path[i],
                path[i + 1],
                currentAmount
            );
        }

        uint256 amountOut = currentAmount;

        // 3. Проверка slippage
        if (amountOut < minAmountOut) {
            revert SlippageExceeded(amountOut, minAmountOut);
        }

        // 4. Отправка результата пользователю
        IERC20(path[path.length - 1]).safeTransfer(msg.sender, amountOut);

        emit SwapExecuted(msg.sender, path[0], path[path.length - 1], amountIn, am
ountOut);
    }

    /**
     * @dev Выполняет одиночный своп на пуле Uniswap V2.
     */
    function _swapUniswapV2(
        address pool,
        address tokenIn,
        address tokenOut,
        uint256 amountIn
    ) internal returns (uint256 amountOut) {
        // Определяем, является ли tokenIn токеном0 или токеном1 пула
        address token0 = IUniswapV2Pair(pool).token0();
        (uint256 reserve0, uint256 reserve1) = _getReserves(pool);
```

```
        (uint256 reserveIn, uint256 reserveOut) = tokenIn == token0
            ? (reserve0, reserve1)
            : (reserve1, reserve0);

        // Рассчитываем amountOut по формуле Uniswap V2 (с комиссией 0.3%)
        amountOut = _getAmountOut(amountIn, reserveIn, reserveOut);

        // Даем аппрув пулу
        IERC20(tokenIn).forceApprove(pool, amountIn);

        // Вызываем swap на пуле
        IUniswapV2Pair(pool).swap(
            tokenIn == token0 ? 0 : amountOut,
            tokenIn == token0 ? amountOut : 0,
            address(this),
            new bytes(0)
        );
    }

    // --- Вспомогательные чистые функции (должны быть public для off-chain симуля
ции) ---

    function _getReserves(address pool) internal view returns (uint256 r0, uint256
r1) {
        (uint112 reserve0, uint112 reserve1,) = IUniswapV2Pair(pool).getReserves
();
        return (uint256(reserve0), uint256(reserve1));
    }

    function _getAmountOut(
        uint256 amountIn,
        uint256 reserveIn,
        uint256 reserveOut
    ) internal pure returns (uint256) {
        require(amountIn > 0 && reserveIn > 0 && reserveOut > 0, "Invalid input
s");
        uint256 amountInWithFee = amountIn * 997;
        uint256 numerator = amountInWithFee * reserveOut;
        uint256 denominator = (reserveIn * 1000) + amountInWithFee;
        return numerator / denominator;
    }

    // Для off-chain симуляции можно добавить view функцию, которая возвращает ожи
```

```
    // для off-chain симуляции можно добавить view-функцию, которая возвращает ожи
даемый amountOut
    function getAmountsOut(
        address[] calldata path,
        address[] calldata pools,
        uint256 amountIn
    ) external view returns (uint256 amountOut) {
        uint256 current = amountIn;
        for (uint256 i = 0; i < pools.length; i++) {
            address token0 = IUniswapV2Pair(pools[i]).token0();
            (uint256 r0, uint256 r1) = _getReserves(pools[i]);
            (uint256 rIn, uint256 rOut) = path[i] == token0 ? (r0, r1) : (r1, r0);
            current = _getAmountOut(current, rIn, rOut);
        }
        return current;
    }
}
```

## 🖥️ Минимальный Off-chain Routing Engine (псевдокод на TypeScript)

Движок должен найти путь, который дает максимум `tokenOut`. Для MVP можно использовать простой жадный алгоритм или полный перебор по предзагруженному списку пулов.

```typescript
// Псевдокод для понимания логики, а не готовый к продакшену код
interface PoolInfo {
    address: string;
    token0: string;
    token1: string;
    reserve0: bigint;
    reserve1: bigint;
}

// Функция расчета amountOut для одного пула (аналог солидити)
function getAmountOut(amountIn: bigint, reserveIn: bigint, reserveOut: bigint): bi
gint {
    const amountInWithFee = amountIn * 997n;
    const numerator = amountInWithFee * reserveOut;
    const denominator = (reserveIn * 1000n) + amountInWithFee;
```

```
    return numerator / denominator;
}

// Поиск лучшего маршрута (упрощенный BFS для двух хопов)
async function findBestRoute(tokenIn: string, tokenOut: string, amountIn: bigint):
Promise<Route> {
    const pools: PoolInfo[] = await fetchAllV2Pools(); // Загружаем данные из Subg
raph или RPC

    // 1. Прямой своп
    const directPools = pools.filter(p =>
        (p.token0 === tokenIn && p.token1 === tokenOut) ||
        (p.token1 === tokenIn && p.token0 === tokenOut)
    );

    let bestAmountOut = 0n;
    let bestPath: string[] = [];
    let bestPools: string[] = [];

    for (const pool of directPools) {
        const [resIn, resOut] = getReservesForDirection(pool, tokenIn, tokenOut);
        const out = getAmountOut(amountIn, resIn, resOut);
        if (out > bestAmountOut) {
            bestAmountOut = out;
            bestPath = [tokenIn, tokenOut];
            bestPools = [pool.address];
        }
    }

    // 2. Двух-хоповые маршруты (например, tokenIn -> USDC -> tokenOut)
    // ... (перебор промежуточных токенов)

    return { path: bestPath, pools: bestPools, expectedAmountOut: bestAmountOut };
}

// Подготовка calldata для контракта
function encodeRoute(path: string[], pools: string[]): string {
    const abiCoder = new ethers.AbiCoder();
    return abiCoder.encode(['address[]', 'address[]'], [path, pools]);
}
```

## ✅ Что получаем на выходе?

Эта минимальная связка позволяет:

1. Пользователь вводит `tokenIn`, `tokenOut` и сумму.
2. Фронтенд вызывает off-chain движок, который находит маршрут и вычисляет `expectedAmountOut`.
3. Фронтенд применяет slippage (например, 0.5%) и рассчитывает `minAmountOut`.
4. Фронтенд отправляет транзакцию на `MinimalDEXAggregator.swap()` с полученным `path`, `pools`, `amountIn`, `minAmountOut` и `deadline`.
5. Контракт выполняет своп атомарно, защищая от проскальзывания.

Это **абсолютный минимум**, достаточный для демонстрации концепции и даже для реального использования в тестовой сети. Для продакшена потребуется добавить поддержку Uniswap V3, сплит-маршруты, интеграцию с несколькими фабриками пулов и более сложные алгоритмы защиты от MEV.

Хочешь углубиться в какой-то конкретный компонент, например, в алгоритм поиска маршрута или интеграцию Uniswap V3?

> как рассчитать оптимальный маршрут обмена между двумя токенами через несколько DEX

Thought for 11 seconds

> Пользователь хочет узнать, как рассчитать оптимальный маршрут обмена между двумя токенами через несколько DEX. Мне нужно дать теоретическое обоснование и примеры реализации на Solidity. Следуя плану, я должен охватить несколько аспектов: графовую модель, алгоритмы поиска, сплит-маршруты, особенности Uniswap V3, формулы расчета, оффчейн-реализацию, ончейн-верификацию, оптимизацию газа и MEV-защиту. Для этого мне потребуется собрать информацию из различных источников. Я выполню несколько параллельных поисковых запросов, чтобы охватить все эти темы.

Found 78 web pages

> После выполнения всех поисковых запросов, я получил результаты по каждому из аспектов. Теперь мне нужно проанализировать их и выбрать наиболее релевантные источники для формирования ответа. Я открою несколько наиболее многообещающих результатов, чтобы получить более подробную информацию.

Read 7 pages Что такое 1inch (1INCH) и как это работает? ↗
Auto Router | Uniswap/docs | DeepWiki ↗ GitHub - okxlabs/DEX-Router-Sui-V1 ↗
How the Aggregator Works | Dirol ↗
GitHub - DecenterApps/RateX: Decentralized dex aggregator · GitHub ↗ View All

> Теперь у меня есть достаточно информации для составления ответа. Я структурирую его следующим образом: введение, графовая модель ликвидности, алгоритмы поиска маршрута, сплит-маршруты, особенности Uniswap V3, формулы расчета на Solidity, оффчейн-реализация, ончейн-верификация, оптимизация газа и MEV-защита, заключение. В ответе я буду использовать информацию из найденных источников.

Для DEX-агрегатора поиск оптимального маршрута — ключевая задача. Я подготовил подробный разбор, который объединяет теорию графов, конкретные алгоритмы и код на Solidity.

## 🧩 Теоретическое обоснование: От "Где дешевле?" к "Графу"

Поиск маршрута — это не просто выбор одной биржи с лучшей ценой. Это сложная задача, которая решается с помощью **теории графов**.

### 1. Базовая модель: Построение графа ликвидности

Весь рынок DeFi можно представить как **взвешенный ориентированный граф**.

- **Узлы (Nodes)**: Все токены (ETH, USDC, DAI, WBTC и т.д.) [32].
- **Ребра (Edges)**: Все пулы ликвидности, через которые можно обменять один токен на другой [32].
- **Вес ребра (Edge Weight)**: Обратно пропорционален выгоде обмена.

### 2. Как назначается "вес" (стоимость) маршрута?

Алгоритм не ищет самый короткий путь в "попугаях". Он ищет путь, который при заданном количестве токена А **даст максимальное количество токена Б** на выходе.

Вес ребра — это **эффективность пула**, которая складывается из:

- **Комиссии пула**: Напрямую уменьшает сумму на выходе.
- **Глубины ликвидности**: Влияет на ценовое воздействие (Price Impact). Чем глубже пул, тем меньше ваша сделка сдвинет цену.
- **Ценового воздействия (Slippage)**: Чем крупнее сделка относительно пула, тем

хуже итоговый курс.

Алгоритм должен смоделировать прохождение вашей сделки через каждый потенциальный пул и сравнить конечный результат.

**3. Постановка задачи**

**Входные данные:**

- Токен для продажи ( `tokenIn` )
- Токен для покупки ( `tokenOut` )
- Количество для продажи ( `amountIn` )

**Выходные данные:**

- Оптимальный маршрут: последовательность адресов токенов и пулов ликвидности.
- Максимальное количество токена на выходе ( `maxAmountOut` ).

## 🗺️ Алгоритмы поиска пути: От простого к сложному

Для поиска в таком графе используются классические алгоритмы, адаптированные под специфику DeFi.

**1. Алгоритм Дейкстры**

Это один из базовых алгоритмов для поиска кратчайшего (в нашем случае — самого выгодного) пути в графе.

- **Принцип**: Находит оптимальный путь от стартовой вершины до всех остальных.
- **Применение в DeFi**: Хорошо работает, когда «вес» ребра (эффективность обмена) является фиксированным числом, как в пулах Uniswap V2.
- **Минус**: Не справляется со сложной, динамической структурой Uniswap V3.

**2. Алгоритм Беллмана-Форда (и его модификации)**

Это более мощный и гибкий инструмент, который часто используют продвинутые агрегаторы, такие как 1inch[32].

- **Принцип**: Умеет работать с "отрицательными весами", что в контексте DeFi

позволяет эффективно искать **арбитражные возможности** (пути, проходя которые, вы получаете больше, чем отдали).

- **Применение**: Идеален для поиска сложных, многошаговых маршрутов (до 4-5 обменов), где каждый шаг вносит свой вклад в итоговую цену[32].

## 📈 Сплит-маршруты (Split Routes): Дробление для выгоды

Одна из главных инноваций агрегаторов — это сплит-маршруты[12].

- **Проблема**: При крупной сделке в одном пуле вы сильно сдвигаете цену против себя (высокий Price Impact), что уменьшает итоговую прибыль.
- **Решение**: Агрегатор «дробит» крупную сделку на несколько частей и отправляет их по разным, но тоже выгодным маршрутам.
  - Например, 60% суммы идет через Uniswap V3, а 40% через Curve[32].
- **Результат**: Суммарный ценовой удар по рынку становится меньше, и вы получаете больше токенов на выходе.

## 💻 Практическая реализация: Формулы расчета на Solidity

Сердце любого маршрутизатора — это умение точно рассчитать результат обмена в пуле. Привожу примеры кода на **Solidity 0.8.24**, которые можно использовать в смарт-контракте агрегатора.

### 1. Для Uniswap V2 (Constant Product AMM)

Здесь все просто. Формула для расчета получаемого количества токенов с учетом комиссии 0.3%[56]:

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

library UniswapV2Library {
    /**
     * @dev Рассчитывает результат обмена для пула Uniswap V2.
     * @param amountIn Количество входящего токена.
     * @param reserveIn Резерв входящего токена в пуле.
     * @param reserveOut Резерв исходящего токена в пуле.
```

```
     * @return amountOut Количество исходящего токена.
     */
    function getAmountOut(
        uint256 amountIn,
        uint256 reserveIn,
        uint256 reserveOut
    ) internal pure returns (uint256 amountOut) {
        require(amountIn > 0, "UniswapV2Library: INSUFFICIENT_INPUT_AMOUNT");
        require(reserveIn > 0 && reserveOut > 0, "UniswapV2Library: INSUFFICIENT_L
IQUIDITY");

        uint256 amountInWithFee = amountIn * 997;
        uint256 numerator = amountInWithFee * reserveOut;
        uint256 denominator = (reserveIn * 1000) + amountInWithFee;
        amountOut = numerator / denominator;
    }
}
```

### 2. Для Uniswap V3 (Concentrated Liquidity)

Здесь расчет сложнее, так как сделка может пересекать несколько ценовых "тиков". Это псевдокод, который отражает основную идею:

```solidity
// Примерная логика расчета для Uniswap V3 (сильно упрощено)

function getAmountOutUniswapV3(
    address pool,
    bool zeroForOne,
    uint256 amountIn
) internal returns (uint256 amountOut) {
    (uint160 sqrtPriceX96, int24 currentTick, , , , , ) = IUniswapV3Pool(pool).slo
t0();
    uint128 liquidity = IUniswapV3Pool(pool).liquidity();

    while (amountIn > 0) {
        // Находим следующий ценовой "тик", на котором есть ликвидность
        (int24 nextTick, bool initialized) = getNextTick(pool, currentTick, zeroFo
rOne);

        // Рассчитываем, сколько токенов нужно, чтобы цена дошла до этого тика
        uint256 amountInToNextTick = computeAmountInToReachTick(amountIn, sqrtPric
```

```
eX96, liquidity, nextTick);

        // Если сделка не исчерпывает ликвидность в текущем тике
        if (amountIn < amountInToNextTick) {
            amountOut += computeAmountOutForAmountIn(amountIn, sqrtPriceX96, liqui
dity, zeroForOne);
            amountIn = 0;
        } else {
            // Сделка переходит на следующий тик, обновляем состояние
            amountOut += computeAmountOutForAmountIn(amountInToNextTick, sqrtPrice
X96, liquidity, zeroForOne);
            amountIn -= amountInToNextTick;
            // Обновляем sqrtPriceX96, currentTick и liquidity для следующего тика
            (sqrtPriceX96, currentTick, liquidity) = updateStateForNextTick(pool,
nextTick, liquidity);
        }
    }
}
```

## 🖥️ Off-chain реализация: Интеграция алгоритмов

Эти сложные расчеты выполняются не в блокчейне, а на вашем бэкенде или во фронтенде. Для этого есть готовые инструменты, например **Uniswap Smart Order Router** [12] . Вот как может выглядеть его использование:

```typescript
import { AlphaRouter } from '@uniswap/smart-order-router';
import { Token, CurrencyAmount, TradeType, Percent } from '@uniswap/sdk-core';
import { ethers } from 'ethers';

// Инициализация роутера
const router = new AlphaRouter({ chainId: 1, provider: web3Provider });

// Определение токенов
const WETH = new Token(1, '0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2', 18, 'WET
H');
const USDC = new Token(1, '0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48', 6, 'USD
C');

// Запрос маршрута
const route = await router.route(
```

```
  CurrencyAmount.fromRawAmount(WETH, ethers.utils.parseEther('1').toString()), //
amountIn
  USDC, // tokenOut
  TradeType.EXACT_INPUT,
  {
    recipient: walletAddress,
    slippageTolerance: new Percent(5, 1000), // 0.5%
    deadline: Math.floor(Date.now() / 1000 + 1800), // 30 минут
  }
);

// Результат
console.log(`Лучшая цена: ${route.quote.toFixed(6)} USDC`);
console.log(`Оценка газа: ${route.estimatedGasUsed.toString()}`);
```

## ⛓️ On-chain верификация: Проверка и исполнение

После того как off-chain сервис нашел оптимальный маршрут, он передает его в смарт-контракт для атомарного исполнения.

Вот упрощенная функция `swap`, которая принимает найденный маршрут и выполняет его:

```solidity
function swap(
    address[] calldata path,  // Маршрут токенов: [tokenIn, ..., tokenOut]
    address[] calldata pools, // Адреса пулов для каждого шага
    uint256 amountIn,         // Сумма на входе
    uint256 minAmountOut,     // Защита от проскальзывания
    uint256 deadline          // Крайний срок
) external nonReentrant {
    require(block.timestamp <= deadline, "Expired");

    IERC20(path[0]).safeTransferFrom(msg.sender, address(this), amountIn);
    uint256 currentAmount = amountIn;

    // Проходим по маршруту и вызываем каждый пул
    for (uint256 i = 0; i < pools.length; i++) {
        currentAmount = _swapOnPool(pools[i], path[i], path[i+1], currentAmount);
    }
```

```
    require(currentAmount >= minAmountOut, "Slippage too high");
    IERC20(path[path.length - 1]).safeTransfer(msg.sender, currentAmount);
}
```

## ⚡ Оптимизация и безопасность: Ключевые нюансы

- **Оптимизация газа**: Выполнение сложного маршрута в смарт-контракте стоит денег. Поэтому агрегаторы, такие как **YakSwap**, учитывают стоимость газа в своем алгоритме поиска[73]. Иногда более простой маршрут с чуть худшей ценой, но с меньшими затратами газа, оказывается выгоднее.
- **MEV-защита**: Фронтраннинг-боты могут украсть вашу прибыль. Продвинутые агрегаторы, такие как **1inch** (через режим Fusion) и **CoW Protocol**, используют механизмы вроде "пакетных аукционов" (batch auctions) или "намерений" (intents), чтобы защитить пользователей[1].

## 💎 Заключение

Расчет оптимального маршрута — это сложный, многоступенчатый процесс, который является сердцем любого DEX-агрегатора. Он требует комбинации глубоких знаний теории графов, особенностей различных AMM и навыков как off-chain, так и on-chain разработки.

Надеюсь, этот детальный разбор поможет вам в создании вашего проекта. Если появятся новые вопросы — смело задавайте.

Если какой-то из аспектов нужно раскрыть подробнее, дайте знать.

78 web pages

как учитывать комиссии DEX при расчёте лучшего маршрута

Thought for 5 seconds

> Мы должны объяснить, как учитывать комиссии DEX при расчете оптимального маршрута в агрегаторе. Нужно теоретическое обоснование и практическая реализация в Solidity 0.8.24 с OpenZeppelin 5.0+. Следует рассмотреть типы комиссий: фиксированные процентные (Uniswap V2), переменные уровни (Uniswap V3), комиссии протокола, и

> процентные (Uniswap V2), переменные уровни (Uniswap V3), комиссии протокола, и возможные комиссии за газ. Также важно показать, как агрегатор сравнивает маршруты с разными комиссиями, как комиссии влияют на amountOut, и как это встраивается в алгоритм поиска пути. В коде показать расчет с комиссией на примере Uniswap V2 и V3.

## 🧩 Теоретическое обоснование: Роль комиссий в агрегаторе

При поиске лучшего маршрута агрегатор должен учитывать **все виды комиссий**, которые взимаются на каждом шаге обмена. Игнорирование даже небольшой комиссии может привести к выбору неоптимального маршрута, особенно если сумма сделки велика или разница в комиссиях между пулами существенна.

### Основные типы комиссий в DEX:

| Тип комиссии | Пример | Влияние на агрегатор |
|---|---|---|
| **Комиссия пула (LP fee)** | Uniswap V2: 0.3%, Uniswap V3: 0.05% / 0.3% / 1% | Напрямую уменьшает `amountOut` на каждом шаге. |
| **Комиссия протокола (Protocol fee)** | Некоторые DEX берут долю от LP fee (например, Uniswap V3 включает `feeProtocol`). | Обычно уже включена в LP fee с точки зрения трейдера. |
| **Газ (Gas fee)** | Стоимость исполнения транзакции в сети Ethereum. | Влияет на **итоговую экономическую выгоду**. Маршрут с чуть лучшей ценой, но с высоким газом может быть убыточным. |
| **Комиссия агрегатора** | Плата, взимаемая самим агрегатором (например, часть `amountIn` или доля от `amountOut`). | Учитывается при расчете `minAmountOut`. |

### Как комиссии влияют на алгоритм поиска пути?

При построении графа ликвидности каждое ребро (пул) имеет **эффективную цену**

**исполнения**, которая зависит от:

1. Резервов пула (влияет на Price Impact).
2. **Комиссии пула** (чем выше комиссия, тем меньше токенов получит трейдер при прочих равных).

В агрегаторах типа 1inch или Matcha алгоритм маршрутизации (например, Беллман-Форд) ищет путь, который максимизирует `amountOut` **после вычета всех комиссий пула**, а затем сравнивает итоговую сумму с учетом стоимости газа.

## 🧮 Математический учет комиссий

### 1. Uniswap V2 (фиксированная комиссия 0.3%)

Формула расчета `amountOut` уже включает комиссию через множитель `997/1000`:

```solidity
amountOut = (amountIn * 997 * reserveOut) / (reserveIn * 1000 + amountIn * 997)
```

Здесь комиссия "зашита" в коэффициенте `997` (0.3% от `amountIn` удерживается).

### 2. Uniswap V3 (переменная комиссия)

В V3 комиссия задается в **сотых долях базисного пункта** (1 базисный пункт = 0.01%, 1 сотня базисных пунктов = 1%).
Значение `fee` в контракте пула — это, например, `3000` для 0.3% или `500` для 0.05%.
При локальном расчете (off-chain) агрегатор должен использовать именно это значение для корректного вычисления `amountOut` на каждом тике.

Упрощенная формула для одного тика (без пересечения границ):

```solidity
// fee = 3000 (0.3%)
uint256 amountInAfterFee = amountIn * (1_000_000 - fee) / 1_000_000;
// Далее стандартная формула константного произведения внутри тика
```

## 💻 Практическая реализация в Solidity (0.8.24 + OZ 5.0+)

В смарт-контракте агрегатора комиссии учитываются **автоматически**, потому что мы либо вызываем функцию `swap` на пуле, которая сама применяет комиссию, либо локально вычисляем `amountOut` по формулам с учетом комиссии (для проверки `minAmountOut`). Но при **поиске маршрута off-chain** мы должны явно применять комиссии.

Рассмотрим пример библиотеки для работы с разными типами пулов, где комиссия является параметром.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

library DexLibrary {
    uint256 constant FEE_DENOMINATOR = 1_000_000; // Для точности V3

    /**
     * @dev Расчет amountOut для Uniswap V2 с фиксированной комиссией 0.3%.
     */
    function getAmountOutV2(
        uint256 amountIn,
        uint256 reserveIn,
        uint256 reserveOut
    ) internal pure returns (uint256) {
        uint256 amountInWithFee = amountIn * 997; // 0.3% комиссия
        uint256 numerator = amountInWithFee * reserveOut;
        uint256 denominator = (reserveIn * 1000) + amountInWithFee;
        return numerator / denominator;
    }

    /**
     * @dev Расчет amountOut для Uniswap V3 с заданной комиссией (например, 3000 = 0.3%).
     *      Упрощенно для одного тика. В реальности нужен итеративный проход по тикам.
     */
    function getAmountOutV3SingleTick(
        uint256 amountIn,
        uint160 sqrtPriceX96,
        uint128 liquidity,
```

```
        uint24 fee,
        bool zeroForOne
    ) internal pure returns (uint256 amountOut) {
        // 1. Вычисляем amountIn после вычета комиссии
        uint256 amountInAfterFee = amountIn * (FEE_DENOMINATOR - fee) / FEE_DENOMI
NATOR;

        // 2. Применяем формулы Uniswap V3 для расчета amountOut
        //    (детали опущены для краткости, но суть: комиссия применяется до осно
вного расчета)
        // ...
        return amountOut;
    }

    /**
     * @dev Универсальная функция для расчета amountOut с учетом комиссии пула.
     *      В реальном агрегаторе здесь будет диспетчеризация по типу DEX.
     */
    function getAmountOut(
        address pool,
        uint256 amountIn,
        address tokenIn,
        address tokenOut
    ) internal view returns (uint256) {
        // Проверяем тип пула (например, по наличию функции fee() или factory)
        // Для V2 используем getAmountOutV2
        // Для V3 извлекаем fee, sqrtPriceX96, liquidity и вызываем getAmountOutV3
    }
}
```

## 🔍 Off-chain алгоритм поиска с учетом комиссий

В движке маршрутизации (TypeScript/Python) каждое ребро графа хранит информацию о комиссии. При расчете стоимости прохода через ребро применяется соответствующая формула.

Пример на псевдокоде TypeScript:

```typescript
interface PoolEdge {
    address: string;
```

```
    tokenIn: string;
    tokenOut: string;
    dexType: 'UniswapV2' | 'UniswapV3';
    reserveIn: bigint;
    reserveOut: bigint;
    fee?: number; // для V3: 500, 3000, 10000
}

function computeAmountOut(amountIn: bigint, edge: PoolEdge): bigint {
    if (edge.dexType === 'UniswapV2') {
        const amountInWithFee = amountIn * 997n;
        const numerator = amountInWithFee * edge.reserveOut;
        const denominator = (edge.reserveIn * 1000n) + amountInWithFee;
        return numerator / denominator;
    } else if (edge.dexType === 'UniswapV3') {
        const feeDenom = 1_000_000n;
        const amountInAfterFee = amountIn * (feeDenom - BigInt(edge.fee!)) / feeDe
nom;
        // Далее расчет через тики с использованием amountInAfterFee
        // ...
    }
    return 0n;
}
```

## ⛽ Учет стоимости газа (Gas Cost)

Газ — это тоже комиссия, которая может сделать маршрут убыточным. Агрегатор должен оценивать gas usage для каждого потенциального маршрута и корректировать итоговую выгоду.

**Упрощенный подход:**

1. Для каждого шага маршрута (вызова пула) оцениваем газ (например, ~100k газа на своп V2, ~150k на V3).
2. Умножаем на текущую цену газа ( `gasPrice` ) и цену нативного токена ( `ETH/USD` ).
3. Вычитаем стоимость газа из `amountOut` , конвертированного в USD (или добавляем к `amountIn` для корректного сравнения).

**Формула "чистой выгоды":**

```text
netAmountOut = amountOut - (estimatedGas * gasPrice * ethPrice) / tokenOutPrice
```

Агрегатор выбирает маршрут с максимальным `netAmountOut`.

## 🧪 Пример: сравнение двух маршрутов с разными комиссиями

Предположим, у нас есть два пути для обмена 1 ETH на USDC:

**Маршрут 1: Прямой своп через Uniswap V3 (комиссия 0.05%)**

- `amountOut` = 2000 USDC
- Газ = 150,000

**Маршрут 2: ETH → WBTC → USDC через два пула Uniswap V2 (комиссия 0.3% каждый)**

- `amountOut` = 2005 USDC (казалось бы, лучше)
- Газ = 250,000 (больше вызовов)

При цене газа $50 за транзакцию:

- Маршрут 1: чистая выгода ≈ 2000 - (150k * 50 / 1e9) ≈ 2000 - 0.0075 = **1999.99 USDC**
- Маршрут 2: чистая выгода ≈ 2005 - (250k * 50 / 1e9) ≈ 2005 - 0.0125 = **2004.99 USDC**

В итоге Маршрут 2 все равно выгоднее, несмотря на более высокие комиссии и газ. Агрегатор должен уметь выполнять такие сравнения.

## ⚠️ Нюансы безопасности и точности

1. **Погрешность округления**: В Solidity деление целочисленное с отбрасыванием остатка. При последовательных вычислениях на off-chain стороне нужно эмулировать такое же поведение, чтобы результат совпадал с реальным исполнением в блокчейне.

исполнением в блокчейне.

2. **Динамическое изменение комиссий**: В Uniswap V3 комиссия пула фиксирована при создании, но в некоторых протоколах (например, Curve) комиссия может меняться голосованием DAO. Агрегатор должен периодически обновлять кэш данных.
3. **Учет protocol fee**: Хотя protocol fee не влияет на расчет `amountOut` для трейдера, она уменьшает резервы пула со временем. Для точного моделирования агрегатору нужно использовать **реальные резервы** пула, которые уже учитывают накопленный protocol fee.

## 📌 Вывод

Учет комиссий — это не просто подстановка числа в формулу, а многофакторная оптимизация, включающая:

- Процентные комиссии каждого пула в маршруте.
- Стоимость газа для выполнения всех шагов.
- Влияние размера сделки на цену (Price Impact) через глубину ликвидности.

Именно поэтому современные агрегаторы используют сложные off-chain алгоритмы (например, модифицированный Беллман-Форд с эвристиками), которые оценивают **чистую экономическую выгоду** с учетом всех видов издержек.

Если потребуется более детально разобрать реализацию конкретного этапа (например, расчет газа или интеграцию с пулами Curve), дайте знать.

почему нужно мокать контракты для тестирования вместо настоящего uniswap

Thought for 5 seconds

> Мы перешли от теоретических вопросов к практическому тестированию смарт-контрактов. Пользователь спрашивает, зачем мокать контракты при тестировании агрегатора, вместо использования реальных контрактов Uniswap. Нужно объяснить причины с технической и экономической точек зрения, а также показать примеры моков на Solidity/TypeScript. Ответ должен быть структурирован: теоретическое обоснование и практическая

> реализация. Важно подчеркнуть специфику тестирования агрегатора: контроль над состоянием пулов, воспроизводимость, изоляция, скорость, тестирование крайних случаев. Приведу пример мок-контракта на Solidity для UniswapV2Pair и пример теста на Foundry или Hardhat.

## 🧪 Теоретическое обоснование: Зачем мокать DeFi-протоколы при тестировании агрегатора

Использование реальных контрактов Uniswap (даже на форке основной сети) для тестирования **логики самого агрегатора** — это распространённая, но часто неоптимальная практика. Моки (mock-контракты) решают несколько фундаментальных проблем, специфичных для DEX-агрегаторов.

### 1. Детерминизм и воспроизводимость тестов

Реальные пулы Uniswap имеют динамическое состояние: резервы меняются с каждым блоком, появляются новые пулы, комиссии могут варьироваться. Это делает тесты **недетерминированными**.

- **Пример**: Вы написали тест, который ожидает `amountOut = 1000`. На момент написания это было верно. Через неделю кто-то свопнул огромную сумму в этом пуле, резервы изменились, и ваш тест начал падать, хотя код агрегатора не менялся.
- **Решение**: Мок-контракт возвращает **фиксированные, заранее известные значения**. Вы полностью контролируете среду тестирования.

### 2. Тестирование граничных условий и ошибок

С помощью моков можно легко смоделировать ситуации, которые в реальной сети поймать сложно или дорого:

- Пул **без ликвидности** (reserve = 0).
- Пул с **огромным проскальзыванием**.
- Пул, который **ревертит транзакцию** (например, из-за внутренней ошибки).
- Токены с **transfer tax** (комиссия на перевод).
- Пулы **Uniswap V3** с определенным распределением ликвидности по тикам.

### 3. Скорость выполнения тестов

Форк основной сети (Foundry `vm.createSelectFork` или Hardhat `hardhat_reset`)

требует скачивания состояния блокчейна и выполнения вычислений на реальных контрактах. Для Uniswap V3 это **очень ресурсоёмко**, так как контракты большие и содержат сложные математические операции с 256-битными числами.

- **Результат**: Набор тестов может идти минуты вместо секунд. Моки выполняются мгновенно, так как это простые заглушки.

#### 4. Изоляция тестируемой логики

Ваша задача — проверить, правильно ли **агрегатор** собирает маршруты, считает суммы и управляет токенами. Вы **не** тестируете математику Uniswap (она уже протестирована командой Uniswap).

- **Проблема**: Если тест упал с реальным Uniswap, вы не знаете: это баг в вашем коде или изменилось состояние пула.
- **Решение**: Мок-контракт исключает внешние зависимости. Если тест падает — виноват ваш код.

#### 5. Экономия средств и лимитов RPC

Бесплатные RPC-провайдеры (Alchemy, Infura) имеют лимиты на количество запросов. Интенсивное тестирование с форками быстро исчерпывает эти лимиты. Локальное тестирование с моками не требует внешних запросов.

#### 6. Настройка сложных сценариев MEV

Чтобы протестировать защиту от сэндвич-атак или проверку `deadline`, нужно манипулировать временем блока (`vm.warp`), изменять резервы пула прямо в процессе транзакции или имитировать фронтраннинг. С моками это сделать тривиально через `vm.mockCall`. С реальным контрактом Uniswap вам пришлось бы писать сложные сценарии с ботом, что сильно усложняет тесты.

## 🛠️ Практическая реализация: Мок-контракты для Uniswap V2 и V3

Вот как выглядят минимальные моки для тестирования агрегатора в Foundry (Solidity) и Hardhat (TypeScript).

#### 1. Мок-контракт Uniswap V2 (Solidity для Foundry)

Этот контракт имитирует `IUniswapV2Pair`, позволяя задавать резервы и возвращать вычисленное количество токенов по формуле V2.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

/**
 * @title MockUniswapV2Pair
 * @dev Имитирует поведение пула Uniswap V2 для тестирования агрегатора.
 *      Позволяет контролировать резервы и проверять, какие суммы были отправлены в своп.
 */
contract MockUniswapV2Pair {
    address public token0;
    address public token1;

    uint112 public reserve0;
    uint112 public reserve1;
    uint32 public blockTimestampLast;

    // Для проверки в тестах: какие параметры передавались в swap
    address public lastCaller;
    uint256 public lastAmount0Out;
    uint256 public lastAmount1Out;
    address public lastRecipient;

    constructor(address _token0, address _token1) {
        token0 = _token0;
        token1 = _token1;
    }

    /**
     * @dev Установка резервов пула (вызывается из теста для настройки сценария).
     */
    function setReserves(uint112 _reserve0, uint112 _reserve1) external {
        reserve0 = _reserve0;
        reserve1 = _reserve1;
        blockTimestampLast = uint32(block.timestamp);
    }
```

```
    /**
     * @dev Имитация свопа по формуле Uniswap V2.
     */
    function swap(
        uint256 amount0Out,
        uint256 amount1Out,
        address to,
        bytes calldata /* data */
    ) external {
        // Сохраняем параметры для последующей проверки в тесте
        lastCaller = msg.sender;
        lastAmount0Out = amount0Out;
        lastAmount1Out = amount1Out;
        lastRecipient = to;

        // Проверяем, что своп корректен по формуле (опционально, можно просто пер
evести токены)
        // В реальном тесте вы можете захотеть проверить, что amountOut соответств
ует ожидаемому.

        // Отправляем токены получателю (предполагаем, что у мока есть баланс этих
токенов)
        if (amount0Out > 0) {
            IERC20(token0).transfer(to, amount0Out);
        }
        if (amount1Out > 0) {
            IERC20(token1).transfer(to, amount1Out);
        }
    }

    /**
     * @dev Возвращает резервы в формате, идентичном Uniswap V2.
     */
    function getReserves() external view returns (uint112, uint112, uint32) {
        return (reserve0, reserve1, blockTimestampLast);
    }

    // ... остальные функции интерфейса при необходимости (mint, burn, sync) ...
}
```

### Пример использования в Foundry тесте:

```
 1. ...
```

```solidity
function test_SwapWithMock() public {
    // 1. Деплоим мок-пул
    MockUniswapV2Pair mockPool = new MockUniswapV2Pair(address(tokenA), address(tokenB));

    // 2. Настраиваем резервы (1000 A и 2000 B)
    mockPool.setReserves(1000 ether, 2000 ether);

    // 3. Минтим токены агрегатору и вызываем функцию swap
    //    (внутри она вызовет mockPool.swap)

    // 4. Проверяем, что агрегатор правильно посчитал amountOut и передал верные параметры
    assertEq(mockPool.lastAmount1Out(), expectedAmountOut);
}
```

## 2. Мок-контракт Uniswap V3 (Solidity)

Uniswap V3 сложнее, так как использует `sqrtPriceX96` и `liquidity`. Для тестирования агрегатора часто достаточно замокать только те функции, которые он вызывает (`swap`, `slot0`, `liquidity`).

```solidity
contract MockUniswapV3Pool {
    address public token0;
    address public token1;
    uint24 public fee;

    uint160 public sqrtPriceX96;
    int24 public tick;
    uint128 public liquidity;

    // Для проверки параметров свопа
    bool public zeroForOneCalled;
    int256 public amountSpecifiedCalled;
    uint160 public sqrtPriceLimitX96Called;

    constructor(address _token0, address _token1, uint24 _fee) {
        token0 = _token0;
        token1 = _token1;
```

```
        fee = _fee;
    }

    function setState(uint160 _sqrtPriceX96, int24 _tick, uint128 _liquidity) exte
rnal {
        sqrtPriceX96 = _sqrtPriceX96;
        tick = _tick;
        liquidity = _liquidity;
    }

    function swap(
        address recipient,
        bool zeroForOne,
        int256 amountSpecified,
        uint160 sqrtPriceLimitX96,
        bytes calldata /* data */
    ) external returns (int256 amount0, int256 amount1) {
        zeroForOneCalled = zeroForOne;
        amountSpecifiedCalled = amountSpecified;
        sqrtPriceLimitX96Called = sqrtPriceLimitX96;

        // Упрощённый расчёт: просто возвращаем фиксированное значение или вычисля
ем по формуле
        // В тестах можно заранее задать, что должен вернуть этот мок.
        // Либо можно реализовать упрощённую логику свопа.

        // Допустим, мы ожидаем exactInput (amountSpecified > 0)
        uint256 amountOut = calculateMockAmountOut(uint256(amountSpecified));

        // Переводим токены (предполагаем, что у мока есть баланс)
        if (zeroForOne) {
            IERC20(token1).transfer(recipient, amountOut);
            return (amountSpecified, -int256(amountOut));
        } else {
            IERC20(token0).transfer(recipient, amountOut);
            return (-int256(amountOut), amountSpecified);
        }
    }

    function slot0() external view returns (uint160, int24, uint16, uint16, uint1
6, uint8, bool) {
        return (sqrtPriceX96, tick, 0, 0, 0, 0, false);
    }
```

```
    // ... другие функции при необходимости ...
}
```

### 3. Альтернативный подход: Моки на уровне вызовов через `vm.mockCall` (Foundry)

В Foundry можно вообще не деплоить отдельные мок-контракты, а использовать **Cheatcodes** для подмены ответов от реальных адресов.

```solidity
function test_MockCallUniswapV2() public {
    address mockPool = address(0x123);

    // Мокаем вызов getReserves()
    vm.mockCall(
        mockPool,
        abi.encodeWithSelector(IUniswapV2Pair.getReserves.selector),
        abi.encode(uint112(1000 ether), uint112(2000 ether), uint32(block.timestam
p))
    );

    // Мокаем вызов swap() - мы ожидаем, что он будет вызван с определёнными парам
етрами
    vm.expectCall(
        mockPool,
        abi.encodeWithSelector(IUniswapV2Pair.swap.selector, 0, expectedAmountOut,
address(aggregator), "")
    );

    // Теперь при вызове агрегатора, когда он обратится к mockPool, он получит зам
оканные данные.
}
```

## 📋 Когда всё-таки стоит использовать форк реальной сети?

Моки идеальны для **юнит-тестов** и **интеграционных тестов** вашего агрегатора. Однако для **end-to-end тестов** перед деплоем полезно запустить тесты на форке основной сети. Это позволит:

- Убедиться, что ваш код совместим с **реальными байт-кодами** контрактов Uniswap.
- Проверить взаимодействие с **различными версиями** пулов (например, пулы V2, созданные разными фабриками).
- Обнаружить неочевидные проблемы, связанные с transfer tax токенов или особенностями реализации `swap`.

**Рекомендуемая стратегия:**

1. **Юнит-тесты**: Используйте **моки** для быстрой проверки логики расчета маршрутов, обработки ошибок и учета комиссий.
2. **Интеграционные тесты**: Используйте **мок-контракты** для тестирования взаимодействия вашего роутера с пулами (проверка правильности передачи параметров).
3. **E2E / Дымовые тесты**: Один раз перед релизом прогоните тесты на **форке** мейннета, чтобы убедиться в отсутствии сюрпризов.

## 🎯 Вывод

Мокирование контрактов Uniswap при тестировании DEX-агрегатора — это не просто "хорошая практика", а **необходимость** для создания быстрых, надёжных и предсказуемых тестов. Это позволяет изолировать вашу бизнес-логику, тестировать граничные случаи и значительно ускорить цикл разработки.

Если нужно детальнее разобрать, как мокать конкретные вызовы (например, `quoteExactInputSingle` в Uniswap V3) или настроить окружение для тестирования с моками в Hardhat, дайте знать.

какие тесты написать для проверки работы аггрегатора

Thought for 5 seconds

> Мы должны предложить набор тестов для проверки работы DEX агрегатора, учитывая, что это проект смарт-контракта на Solidity и off-chain части. Нужно охватить юнит-тесты, интеграционные, edge cases, безопасность, а также тестирование off-chain роутера.
>
> Ответ должен быть структурирован: сначала теория (почему тестирование критично),

> затем конкретные категории тестов с примерами на Foundry (или Hardhat), включая моки и форки. Учитываем, что агрегатор взаимодействует с Uniswap V2/V3, возможны split routes, slippage protection и т.д.
>
> Формат ответа: сначала теоретическое обоснование, потом практическая реализация (список тестов с пояснениями и кодом).

## 🧪 Теоретическое обоснование: Стратегия тестирования DEX-агрегатора

Тестирование агрегатора — это не просто проверка "работает или нет". Это комплексная задача, разделённая на несколько уровней, каждый из которых проверяет свой слой абстракции. Без должного покрытия тестами агрегатор рискует потерять средства пользователей из-за неучтённого проскальзывания, неправильного расчёта маршрута или атак повторного входа.

**Пирамида тестирования для агрегатора:**

1. **Юнит-тесты (Unit Tests)** — проверка отдельных чистых функций: расчёт `amountOut`, кодирование/декодирование маршрута, проверка slippage.
2. **Интеграционные тесты с моками (Mock Integration Tests)** — проверка взаимодействия смарт-контракта агрегатора с замоканными версиями пулов.
3. **Интеграционные тесты на форке (Fork Tests)** — проверка совместимости с реальными контрактами Uniswap и актуальным состоянием сети.
4. **Симуляционные тесты Off-chain роутера** — проверка алгоритмов поиска маршрута на исторических данных.
5. **Тесты безопасности и граничных условий** — проверка защиты от атак и некорректных входных данных.
6. **Газовые тесты (Gas Profiling)** — оценка стоимости исполнения различных маршрутов.

## 📋 Категории тестов и конкретные примеры (Solidity + Foundry)

Ниже приведён детальный план тестирования с примерами кода на Foundry. Для Hardhat логика будет аналогичной, но с использованием TypeScript и `ethers.js`.

### 1. Юнит-тесты библиотек и внутренних функций

Эти тесты проверяют математику расчёта свопов и кодирования маршрута. Они быстрые и не требуют деплоя контрактов.

#### Пример: тестирование `getAmountOut` для Uniswap V2

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import "forge-std/Test.sol";
import "../src/libraries/UniswapV2Library.sol";

contract UniswapV2LibraryTest is Test {
    function test_GetAmountOut_Basic() public {
        uint256 amountIn = 1000 ether;
        uint256 reserveIn = 1_000_000 ether;
        uint256 reserveOut = 2_000_000 ether;

        uint256 expected = (amountIn * 997 * reserveOut) / (reserveIn * 1000 + amo
untIn * 997);
        uint256 actual = UniswapV2Library.getAmountOut(amountIn, reserveIn, reserv
eOut);
        assertEq(actual, expected);
    }

    function test_GetAmountOut_RevertIfZeroLiquidity() public {
        vm.expectRevert("UniswapV2Library: INSUFFICIENT_LIQUIDITY");
        UniswapV2Library.getAmountOut(100, 0, 100);
    }

    function test_GetAmountOut_SmallAmount() public {
        // Проверяем, что даже при микроскопическом amountIn возвращается >0
        uint256 amountOut = UniswapV2Library.getAmountOut(1, 1000 ether, 1000 ethe
r);
        assertTrue(amountOut > 0);
    }
}
```

### 2. Интеграционные тесты с мок-контрактами

Здесь мы деплоим реальный контракт агрегатора и подключаем к нему мок-пулы. Проверяем, что агрегатор правильно вызывает `swap` на пулах, обрабатывает передачу токенов и проверяет `minAmountOut`.

### Пример: проверка успешного свопа и события

```solidity
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import "forge-std/Test.sol";
import "../src/DEXAggregator.sol";
import "./mocks/MockUniswapV2Pair.sol";
import "./mocks/MockERC20.sol";

contract DEXAggregatorMockTest is Test {
    DEXAggregator aggregator;
    MockERC20 tokenA;
    MockERC20 tokenB;
    MockUniswapV2Pair mockPool;

    address user = address(0x123);

    function setUp() public {
        aggregator = new DEXAggregator();
        tokenA = new MockERC20("Token A", "TKA", 18);
        tokenB = new MockERC20("Token B", "TKB", 18);
        mockPool = new MockUniswapV2Pair(address(tokenA), address(tokenB));

        // Минтим токены пользователю
        tokenA.mint(user, 1000 ether);
        // Даём аппрув агрегатору
        vm.prank(user);
        tokenA.approve(address(aggregator), type(uint256).max);
    }

    function test_Swap_Success() public {
        // Настраиваем мок-пул
        mockPool.setReserves(1000 ether, 2000 ether);

        // Минтим токен B мок-пулу, чтобы он мог их отдать при свопе
        tokenB.mint(address(mockPool), 2000 ether);
```

```
        address[] memory path = new address[](2);
        path[0] = address(tokenA);
        path[1] = address(tokenB);

        address[] memory pools = new address[](1);
        pools[0] = address(mockPool);

        uint256 amountIn = 100 ether;
        uint256 expectedOut = 166 ether; // примерное значение
        uint256 minAmountOut = expectedOut * 99 / 100; // 1% slippage
        uint256 deadline = block.timestamp + 100;

        vm.prank(user);
        vm.expectEmit(true, true, true, true);
        emit Swapped(user, address(tokenA), address(tokenB), amountIn, expectedOu
t);

        aggregator.swap(path, pools, amountIn, minAmountOut, deadline);

        // Проверяем, что агрегатор вызвал swap на пуле с правильными параметрами
        assertEq(mockPool.lastCaller(), address(aggregator));
        assertEq(mockPool.lastAmount0Out(), 0);
        assertEq(mockPool.lastAmount1Out(), expectedOut);
        assertEq(mockPool.lastRecipient(), address(aggregator));

        // Проверяем балансы
        assertEq(tokenA.balanceOf(user), 900 ether);
        assertEq(tokenB.balanceOf(user), expectedOut);
    }

    function test_Swap_RevertIfSlippageExceeded() public {
        mockPool.setReserves(1000 ether, 2000 ether);
        tokenB.mint(address(mockPool), 2000 ether);

        address[] memory path = new address[](2);
        path[0] = address(tokenA);
        path[1] = address(tokenB);
        address[] memory pools = new address[](1);
        pools[0] = address(mockPool);

        uint256 amountIn = 100 ether;
        // Завышаем minAmountOut, чтобы он был больше реально возможного
```

```
        uint256 impossibleMinAmountOut = 200 ether;

        vm.prank(user);
        vm.expectRevert(abi.encodeWithSelector(DEXAggregator.SlippageExceeded.sele
ctor, uint256(166 ether), impossibleMinAmountOut));
        aggregator.swap(path, pools, amountIn, impossibleMinAmountOut, block.times
tamp + 100);
    }

    function test_Swap_RevertIfDeadlineExpired() public {
        address[] memory path = new address[](2);
        path[0] = address(tokenA);
        path[1] = address(tokenB);
        address[] memory pools = new address[](1);
        pools[0] = address(mockPool);

        vm.prank(user);
        vm.expectRevert(DEXAggregator.DeadlineExpired.selector);
        aggregator.swap(path, pools, 100 ether, 0, block.timestamp - 1);
    }
}
```

## 3. Тесты безопасности и граничных условий

### 3.1. Защита от Reentrancy

Проверяем, что злоумышленник не может повторно войти в функцию `swap` через вредоносный токен или пул.

```solidity
contract MaliciousReceiver {
    DEXAggregator aggregator;
    address[] path;
    address[] pools;
    uint256 amountIn;
    uint256 minAmountOut;
    uint256 deadline;
    uint256 count;

    constructor(DEXAggregator _aggregator) {
        aggregator = _aggregator;
    }
```

```
    function initiateAttack(...) external {
        // Первый вызов swap
        aggregator.swap(path, pools, amountIn, minAmountOut, deadline);
    }

    // Функция, которая будет вызвана при получении токенов (если это ERC777 или ч
ерез fallback)
    receive() external payable {
        if (count == 0) {
            count++;
            aggregator.swap(path, pools, amountIn, minAmountOut, deadline);
        }
    }
}
```

Тест должен убедиться, что второй вызов `swap` ревертится из-за модификатора `nonReentrant`.

### 3.2. Обработка пустых маршрутов и несовпадения длин массивов

```solidity
function test_Swap_RevertIfInvalidPath() public {
    address[] memory path = new address[](1);
    path[0] = address(tokenA);
    address[] memory pools = new address[](0);

    vm.prank(user);
    vm.expectRevert("Invalid path");
    aggregator.swap(path, pools, 100 ether, 0, block.timestamp + 100);
}

function test_Swap_RevertIfPoolsLengthMismatch() public {
    address[] memory path = new address[](3);
    address[] memory pools = new address[](1); // должно быть 2

    vm.prank(user);
    vm.expectRevert("Pools length mismatch");
    aggregator.swap(path, pools, 100 ether, 0, block.timestamp + 100);
}
```

### 3.3. Тестирование с токенами, не возвращающими bool (USDT)

Некоторые токены (как USDT) не возвращают `bool` при `transfer` / `transferFrom`, что ломает стандартный ERC20. Использование `SafeERC20` должно это исправлять. Мок такого токена:

```solidity
contract NonStandardERC20 is MockERC20 {
    function transfer(address to, uint256 amount) public override returns (bool) {
        // Не возвращаем bool (в реальности код просто выполняется без return)
        super.transfer(to, amount);
        // Здесь нет return
    }
}
```

Тест проверяет, что `safeTransfer` не ревертит.

### 4. Интеграционные тесты на форке основной сети

Эти тесты проверяют, что агрегатор корректно работает с реальными контрактами Uniswap. Рекомендуется запускать их периодически (не при каждом коммите).

### Пример: своп WETH → DAI через Uniswap V2

```solidity
contract DEXAggregatorForkTest is Test {
    using SafeERC20 for IERC20;

    DEXAggregator aggregator;
    address constant WETH = 0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2;
    address constant DAI = 0x6B175474E89094C44Da98b954EedeAC495271d0F;
    address constant UNISWAP_V2_WETH_DAI = 0xA478c2975Ab1Ea89e8196811F51A7B7Ade33e
B11;

    uint256 mainnetFork;

    function setUp() public {
        mainnetFork = vm.createFork("https://eth-mainnet.g.alchemy.com/v2/YOUR_KE
Y");
        vm.selectFork(mainnetFork);
        aggregator = new DEXAggregator();
    }
```

```
    }

    function test_Fork_SwapWETHforDAI() public {
        // Взять адрес с большим балансом WETH для теста
        address whale = 0xF04a5cC80B1E94C69B48f5ee68a08CD2F09A7c3E;
        vm.startPrank(whale);

        uint256 amountIn = 1 ether;
        IERC20(WETH).forceApprove(address(aggregator), amountIn);

        address[] memory path = new address[](2);
        path[0] = WETH;
        path[1] = DAI;
        address[] memory pools = new address[](1);
        pools[0] = UNISWAP_V2_WETH_DAI;

        uint256 balanceBefore = IERC20(DAI).balanceOf(whale);

        aggregator.swap(path, pools, amountIn, 0, block.timestamp + 100);

        uint256 balanceAfter = IERC20(DAI).balanceOf(whale);
        assertGt(balanceAfter, balanceBefore);

        vm.stopPrank();
    }
}
```

### 5. Тестирование off-chain роутера (на TypeScript)

Здесь мы проверяем, что алгоритм находит действительно лучший маршрут. Используем исторические данные или снэпшот резервов.

```typescript
import { expect } from "chai";
import { findBestRoute } from "../src/routing";
import { poolsFixture } from "./fixtures/pools";

describe("Off-chain Router", () => {
    it("should find direct route when it's optimal", async () => {
        const pools = poolsFixture; // заранее подготовленные данные о пулах
        const route = await findBestRoute("0xWETH", "0xDAI", ethers.parseEther
("1"), pools);
```

```
        expect(route.path).to.deep.equal(["0xWETH", "0xDAI"]);
        expect(route.pools.length).to.equal(1);
        expect(route.expectedAmountOut).to.be.gt(0);
    });

    it("should choose multi-hop route if it yields more output", async () => {
        const route = await findBestRoute("0xTOKEN_A", "0xTOKEN_C", amountIn, pool
sWithPoorDirectPool);
        expect(route.path.length).to.be.greaterThan(2);
        // Проверяем, что amountOut больше, чем у прямого свопа
    });

    it("should handle split routes", async () => {
        const splitRoute = await findBestRouteWithSplits("0xWETH", "0xUSDC", hugeA
mount);
        expect(splitRoute.splits.length).to.be.greaterThan(1);
        // Проверяем, что сумма частей даёт лучший результат
    });
});
```

### 6. Газовые тесты

В Foundry можно использовать `--gas-report` или встроенную функцию `gasleft()`.

```solidity
function test_Gas_SingleHop() public {
    // ... настройка
    uint256 gasBefore = gasleft();
    aggregator.swap(path, pools, amountIn, minAmountOut, deadline);
    uint256 gasUsed = gasBefore - gasleft();
    assertLt(gasUsed, 150_000); // Убедимся, что не превышает ожидаемый лимит
}
```

## 📌 Чек-лист обязательных тестов для агрегатора

| Категория | Что тестируем | Инструменты |
|---|---|---|
| | Библиотеки расчёта | |

| | | |
|---|---|---|
| **Юнит** | AmountOut, кодирования маршрута, проверки slippage | Foundry / Hardhat (чистые функции) |
| **Интеграция с моками** | Вызов `swap` на пулах, передача параметров, обработка ошибок, события | Mock контракты |
| **Безопасность** | Reentrancy, переполнение, нестандартные токены, deadline, нулевые адреса | Вредоносные контракты, `vm.assume` |
| **Форк-тесты** | Совместимость с Uniswap V2/V3, реальная ликвидность | Foundry Fork / Hardhat Mainnet Fork |
| **Off-chain** | Алгоритм маршрутизации, сплиты, учёт комиссий и газа | Jest/Mocha + эмуляция RPC |
| **Газ** | Оптимизация потребления газа | Foundry gas report |

Эта структура обеспечит высокое качество и надёжность вашего агрегатора. Если потребуется детальнее разобрать какой-либо блок (например, тестирование Uniswap V3 с тиками или симуляцию MEV-атак), дайте знать.

PriceFetcher.txt
TXT 4.74KB

напиши latex формулы для uniswapv2 и uniswapv3

Thought for 10 seconds

> Мы должны написать LaTeX формулы для расчетов в Uniswap V2 и Uniswap V3 на основе предоставленного контракта PriceFetcher. Контракт содержит функции getAmountOut для V2 и упрощенную оценку для V3, а также расчет ценового воздействия. Нам нужны математические формулы в LaTeX, соответствующие логике контракта.
>
> Из кода:
>
> - V2: amountOut = (amountIn * 997 * reserveOut) / (reserveIn * 1000 + amountIn * 997)

- Price Impact: (spotPrice - executionPrice) / spotPrice в базисных пунктах (10000)
- Spot price: reserveOut / reserveIn (с масштабированием на 1e18)
- Execution price: amountOut / amountIn

Для V3:

- Используется упрощенный метод: baseOutput из sqrtPriceX96 и затем учет fee.
- feeMultiplier = (1_000_000 - fee*1000) / 1_000_000
- baseOutput = zeroForOne ? (amountIn * 1e12) / (sqrtPrice / 1e6) : (amountIn * sqrtPrice) / 1e12
- Затем amountOut = (baseOutput * feeMultiplier) / 1e18

Формула spot price для V3 из sqrtPriceX96:

- price = (sqrtPriceX96^2) / 2^192
- если isToken0Reference true, то spot = price, иначе spot = 1/price

Нужно написать красивые LaTeX формулы с пояснениями.

Для документации вашего DEX-агрегатора и понимания математической модели, вот набор LaTeX формул, соответствующих реализации в контракте `PriceFetcher`. Они разделены на Uniswap V2 и V3, а также включают расчет ценового воздействия (Price Impact).

## 📐 1. Uniswap V2 (Constant Product AMM)

**Входные параметры:**

- $\Delta x = \text{amountIn}$ — количество продаваемого токена $X$
- $R_x = \text{reserveIn}$ — резерв токена $X$ в пуле
- $R_y = \text{reserveOut}$ — резерв токена $Y$ в пуле
- Комиссия пула = $0.3% \Rightarrow$ коэффициент $f = 0.997$

**Выходное количество токенов (amountOut):**

Формула константного произведения с комиссией:
$$(R_x + f \cdot \Delta x)(R_y - \Delta y) = R_x R_y$$

Решая относительно $\Delta y$ (amountOut):

```latex
```

```latex
\Delta y = \frac{f \cdot \Delta x \cdot R_y}{R_x + f \cdot \Delta x}
```

В коде это выражено целочисленно:

```latex
\text{amountOut} = \frac{997 \cdot \Delta x \cdot R_y}{1000 \cdot R_x + 997 \cdot \Delta x}
```

**Спотовая цена (Spot Price):**

```latex
P_{\text{spot}} = \frac{R_y}{R_x} \quad \text{(цена токена X в единицах токена Y)}
```

В коде используется масштабирование на $10^{18}$ для точности:

```latex
P_{\text{spot}}^{\text{scaled}} = \frac{R_y \times 10^{18}}{R_x}
```

**Цена исполнения (Execution Price):**

```latex
P_{\text{exec}} = \frac{\Delta y}{\Delta x}
```

В масштабированном виде:

```latex
P_{\text{exec}}^{\text{scaled}} = \frac{\Delta y \times 10^{18}}{\Delta x}
```

## 🎯 2. Uniswap V3 (Concentrated Liquidity) – Упрощенная оценка

**Входные параметры:**

- $\Delta x = \text{amountIn}$
- $\sqrt{P} = \text{sqrtPriceX96}$ (квадратный корень из цены в формате Q64.96)
- $\text{fee}$ — комиссия в сотых долях базисного пункта (например, $3000$ для $0.3\%$)
- $\text{zeroForOne} = \text{tokenIn} < \text{tokenOut}$ (направление свопа)

**Базовое выходное количество (до вычета комиссии):**

Используется приближение, основанное на текущей цене $\sqrt{P}$:

Если $\text{zeroForOne}$ (продаем token0 за token1):

```latex
\Delta y_{\text{base}} = \frac{\Delta x \times 10^{12}}{\sqrt{P} / 10^6} = \frac{\Delta x \times 10^{18}}{\sqrt{P}}
```

Иначе (продаем token1 за token0):

```latex
\Delta y_{\text{base}} = \frac{\Delta x \times \sqrt{P}}{10^{12}}
```

**Учет комиссии:**

Комиссия пула $fee$ (например, $3000$) преобразуется в множитель:

```latex
\text{feeMultiplier} = \frac{1\,000\,000 - fee \times 1000}{1\,000\,000}
```

Итоговое выходное количество:

```latex
\Delta y = \frac{\Delta y_{\text{base}} \times \text{feeMultiplier}}{10^{18}}
```

**Спотовая цена из $\sqrt{P}$ (sqrtPriceX96):**

Сначала извлекается цена $P$ в формате с фиксированной точкой:

```latex
P = \frac{(\sqrt{P})^2}{2^{192}} = \left( \frac{\sqrt{P}}{2^{96}} \right)^2
```

Затем, если $\text{isToken0Reference}$ (цена токена0 в токене1):

```latex
P_{\text{spot}} = P
```

Иначе (цена токена1 в токене0):

```latex
P_{\text{spot}} = \frac{1}{P}
```

Для целочисленных расчетов в коде используется приближение с $10^{36}$ для избежания дробей.

## 📊 3. Расчет ценового воздействия (Price Impact)

Price Impact определяется как относительное отклонение цены исполнения от спотовой цены:

```latex
\text{Price Impact} = \frac{P_{\text{spot}} - P_{\text{exec}}}{P_{\text{spot}}}
```

В коде результат выражается в базисных пунктах (1 bps = 0.01%), где `BASIS_POINTS = 10000` (100%):

```latex
\text{PriceImpact}_{\text{bps}} = \frac{P_{\text{spot}} - P_{\text{exec}}}{P_{\tex
```

```
t{spot}}} \times 10\,000
```

Если $P_{\text{exec}} \ge P_{\text{spot}}$ (теоретически невозможно в AMM без внешнего арбитража), Price Impact считается равным 0. При нулевом `amountOut` возвращается максимальное значение `BASIS_POINTS` (100%).

## 📝 Примечание по точности

В контракте `PriceFetcher` для V3 используется **упрощенная оценка** на основе текущей мгновенной цены. В реальном агрегаторе для точного расчета необходимо итеративно проходить по тикам ликвидности, так как крупная сделка может пересечь несколько ценовых диапазонов и изменить `sqrtPriceX96`.

Эти формулы дают теоретическую основу для понимания логики, реализованной в предоставленном коде. Вы можете вставить их в документацию проекта, используя стандартное окружение LaTeX.